IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 21 October 2025, accepted 25 November 2025, date of publication 3 December 2025,
date of current version 9 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3639645

RESEARCH ARTICLE

Base, Instruct, and Fine-Tune: Evaluating LLMs for Cross-Language Flaky Test Detection

AZEEM AHMAD[1] **AND XIN SUN**[2]
[1]Department of Software Engineering, Prince Sultan University, Riyadh 11586, Saudi Arabia
[2]Department of Computer Sciences, Linköping University, 58185 Linköping, Sweden

Corresponding author: Azeem Ahmad (aahmad@psu.edu.sa)

ABSTRACT Flaky tests exhibit non-deterministic outcomes posing a critical challenge to the reliability of continuous integration (CI) pipelines. While prior approaches rely on static token level features and vocabulary-based heuristics, they often fail to generalize across languages and project domains. In this work, we explore the use of Large Language Models (LLMs) for detecting flaky tests across five programming languages: Java, Python, C++, Go, and JavaScript. We evaluate three families of LLMs: `LLaMA3`, `Mistral`, and `DS-Coder`, in their base, instruction-tuned, and fine-tuned forms. Our empirical results demonstrate that fine-tuned LLMs consistently outperform baseline and vocabulary-driven methods, achieving near perfect F1 scores on high resource languages and substantial improvements in low resource settings. Instruction-tuned variants offer notable benefits, especially in early stage adaptation. Moreover, our cross-language evaluation reveals that LLMs trained on one language can moderately generalize to others, with fine-tuning significantly narrowing the performance gap. The findings highlight the practical value of LLMs as language agnostic flaky test detectors and provide guidance on model adaptation strategies for real world CI/CD environments. This work sets the foundation for future research on generalizable and semantically aware testing tools in multilingual software ecosystems.

INDEX TERMS Flaky tests, non-deterministic tests, fine-tuning, software testing, large language models, base models.

I. INTRODUCTION

Developers need confidence that their changes do not affect existing behavior of software system or introduce new defects. To gain that confidence, developers rely on extensive automated testing. When a test fails, the default assumption is that the failure are due to recent code modifications. Yet many failures are unrelated to code changes and instead arise from test flakiness (TF): tests that behave inconsistently in the absence of real defects. In practice, developers spend substantial effort triaging such failures only to discover that the root cause is flakiness in the test suite [1], [2].

In the literature, a commonly adopted definition describes a flaky test as *one that alternates between passing and failing outcomes when no relevant changes are made to the codebase or the test itself*. King et al. formalize this view by noting that flaky tests can yield both outcomes (i.e., pass or fail) under unchanged conditions [3].

In addition to losing productivity, TF erodes trust in automated testing. Large-scale systems continue to exhibit substantial flakiness: at Google, for example, 4.56% of 1.6 million failed tests were identified as flaky [4]. Other studies report that flaky test suites account for 13% and up to 50% of build failures, respectively [5], [6], and that roughly one in seven tests at Google can fail due to flakiness [7]. Such intermittent, code-independent failures reduce developers' confidence, leading either to ignored failures or to unnecessary debugging efforts that distract from genuine defects [2].

Research on TF spans several directions: empirical analyses of open-source projects and test smells [4], [8], tools to surface or mitigate smell categories linked to flakiness [9], [10], [11], [12], machine learning approaches for prediction [3], and work that elevates TF as a community

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

concern [13]. Root causes span test code, production code, and test infrastructure, with the majority traced to issues in test code particularly test smells that signal deeper maintainability and reliability problems [2].

Our prior study [14] introduced a cross-language perspective on flaky test detection by analyzing test cases written in five languages (e.g., Java, Python, C++, Go, and JavaScript) by holding the experimental protocol constant across (1) within same language, (2) multi-language, and (3) cross-language settings. Rather than limiting features to a single factor, we examined whether training on a heterogeneous pool of languages improves robustness and generalization when predicting flaky versus non-flaky tests. This design allowed us to probe language agnostic signals of flakiness and to quantify how exposure to diverse codebases affects predictability.

A similar work by Pinto et al. focus only on Java and show that particular lexical terms correlate with flakiness [15]. Camara et al. [16] replicate those vocabulary driven findings, reinforcing the connection between test lexicon and flakiness, but do not evaluate cross-language generalizability [16] as we did on our previous work [15]. Our work differs in scope and objective: rather than asking whether vocabulary predicts flakiness in one language, we ask whether models trained on multi-language data can capture shared patterns that transcend language specific syntax and libraries, thereby improving predictability to previously unseen languages.

A. WHY BRING LLMs INTO FLAKY TEST DETECTION?

Our prior work [14] established strong baselines using classic machine learning (ML) models (e.g., Logistic Regression, Random Forest, SVM) trained on TF–IDF features, with within language, multi-language, and cross-language settings that exposed strengths and limits of vocabulary-driven signals for flaky tests.

We hypothesize that bag-of-words and TF–IDF features overlook higher order code semantics (e.g., oracle design, concurrency and timing behavior) that modern large language models (LLMs) can encode. Prior work [17], [18], [19] on code pretrained Transformers shows that incorporating program structure and signals beyond surface tokens such as data flow graphs or identifier semantics improves downstream understanding tasks, indicating richer representations than lexical co-occurrence alone. LLMs also offer (i) *richer code+text semantics*, via pretraining objectives that align natural and programming languages or model program structure [17], [19]; (ii) *cross-language abstractions* learned from diverse corpora, which are known to support transfer across natural languages and, increasingly, across programming languages [20], [21]; (iii) *promptability* enabling zero-/few-shot use, especially with instruction tuned variants [22], [23]; and (iv) *domain adaptation* through supervised fine-tuning on flaky vs. non-flaky labels, a strategy broadly effective for Transformer models [24]. Consequently, evaluating base, instruct, and fine-tuned LLMs is a natural next step beyond our ML baselines to test whether semantic representations reduce dependence on language-specific vocabularies and yield more robust generalization.

B. CURRENT VS. PRIOR WORK

We keep the *same dataset*, *languages* (Java, Python, C++, Go, JavaScript), and *experimental design* (within-language, multi-language, cross-language splits; metrics; significance testing). The only change is the use of large languages models (LLM): instead of classic ML on TF–IDF, we evaluate base, instruction-tuned, and fine-tuned LLMs from three families, comparing them to our prior ML results. We study three families and four variants per family (12 models total):

- **Llama3**: (i) base; (ii) instruct; (iii) base-fine-tuned on our dataset; (iv) instruct-fine-tuned on our dataset.
- **DeepSeek-Coder** (ds-coder): (i) base; (ii) instruct; (iii) base-fine-tuned; (iv) instruct-fine-tuned.
- **Mistral**: (i) base; (ii) instruct; (iii) base-fine-tuned; (iv) instruct-fine-tuned.

Here, base'' denotes the general pretrained model; instruct denotes the instruction-tuned/chat variant; and fine-tuned refers to supervised adaptation on our flaky vs. non-flaky labels.

This study investigates the following research questions (RQs):

- **RQ1.1 - How effective are LLMs in detecting flaky tests within individual programming languages?**
  We will run within language experiments for Java, Python, C++, Go, and JavaScript, comparing base, instruct, and fine-tuned variants of Llama 3, Mistral, and DeepSeek-Coder. We will report precision, recall, Accuracy and F1-score to characterize per-language strengths.
- **RQ1.2 - What is the impact of combining different programming languages on the predictive accuracy of LLMs?**
  We will incrementally pool training data across languages (bi-/tri-/all-language mixtures) while testing on held-out sets to measure the benefit of linguistic diversity. We will quantify improvements (or regressions) relative to single-language training and highlight which languages act as the best ''sources'' of transferable signal to determine test flakiness.
- **RQ1.3 - To what extent does predictive accuracy vary when training on one language and testing on another?**
  This question investigates the cross-language generalizability of flaky test detectors. Specifically, we evaluate how models trained on one programming language perform when tested on another, using a matrix of language pairs (e.g., train on Java, test on Python). This analysis will reveal how well LLMs capture language-agnostic signals of test flakiness. We will also assess asymmetries in transferability (e.g., Java→C++ vs. C++ →).

- **RQ2 - How does LLMs predictive performance compare with vocabulary-focused findings in our prior work [14]?**
  We will contrast our models against our previous work by examining whether LLMs reduce reliance on language-specific lexicons while improving cross-language performance. This comparison clarifies when semantic representations offer practical advantages over vocabulary-only indicators.
- **RQ3 - What is the statistical significance of performance differences among LLMs in detecting flaky tests, and how do these differences vary across programming languages?**
  In this question, we incorporated a detailed statistical analysis to understand the significance of performance differences across various LLMs and programming languages.

This paper makes the following contributions:

- We perform a comprehensive evaluation of 12 LLM variants (base, instruct, and fine-tuned forms of Llama 3, Mistral, and DeepSeek-Coder) for the task of flaky test prediction across five programming languages: Java, Python, C++, Go, and JavaScript.
- We benchmark LLM performance **within individual languages** (RQ1.1), highlighting how well these models detect flakiness in isolated, language-specific test suites compared to prior classic ML baselines.
- We study the impact of cross-language training data mixtures (RQ1.2), assessing whether combining code from multiple languages improves generalization and identifying which languages contribute most effectively to transfer.
- We construct a full cross-language transfer matrix (RQ1.3), training models on one language and testing on others to evaluate cross-language generalizability and uncover asymmetries in transfer potential.
- We directly compare our LLM based results with vocabulary-based findings from our earlier work [14] (RQ3), analyzing whether LLMs are less reliant on language-specific keywords and more capable of learning semantic, language-agnostic signals of flakiness.
- All experiments are conducted using the same dataset, labeling process, and evaluation protocols as our prior study, ensuring a controlled setting for comparative analysis. Reproducible code and artifacts will be made available to support future research.

The datasets and experiment code used in this study is provided in an open source GitHub repo[1] for reproducibility.

Paper Structure: The remainder of this paper is structured as follows: Section II introduces related work on flaky test detection and large language models. Section III describes our experimental pipeline, model variants, and dataset preparation. Section IV presents the evaluation results, including in-language, cross-language, and statistical significance analyses. Section V offers a discussion on practical and theoretical implications as well as on results. Section VI outlines potential threats to validity, and Section VII concludes the paper and highlights future research directions.

[1] https://github.com/eve76/FCLM

II. RELATED WORK

This section provides the related work in different sub-sections to improve readability regarding flaky test detection, prediction, and root cause analysis.

A. UNDERSTANDING FLAKY TESTS AND THEIR CAUSES

Flaky tests are widely recognized as a persistent and costly challenge in software testing. Ahmad et al. [2] conducted a comprehensive empirical study to uncover the root causes of test flakiness, highlighting factors such as non-deterministic behavior, concurrency issues, and resource contention. Further extending the body of knowledge, Luo et al. [4] and Shi et al. [10] examined flaky test occurrences in large scale industrial systems, with a focus on timing related dependencies and environmental influences.

Building on this foundation, Pinto et al. [15] introduced a taxonomy of flaky tests based on their vocabulary, classifying them into categories such as asynchronous and order dependent tests. This classification was empirically validated by Camara et al. [16] across a diverse set of open-source projects. Their findings align with those of Marinov et al. [25], who emphasized the disruptive effects of flaky tests on regression testing, and Hilton et al. [6], who analyzed the economic and practical costs of test flakiness in continuous integration pipelines.

B. CROSS LANGUAGE GENERALIZATION

Cross language flaky test detection has garnered increasing attention. Ahmad et al. [1], [14] explored this challenge by evaluating traditional machine learning classifiers across multiple languages, including Java, Python, C++, and Go, uncovering common predictive patterns shared across them. In a similar direction, Wendler et al. [26] investigated how dataset diversity influences the generalization capability of flaky test classifiers. Cordy et al. [27] experimented with flaky tests across projects, revealing challenges in replicability and dataset dependency. Additionally, Hilton et al. [6] experimented how cross-project data can enhance classifier robustness, complementing Wendler et al. [26] on using regression test flaky test history.

C. REPRODUCTION AND MITIGATION OF FLAKY TESTS

Reproducing flaky test behavior consistently remains a significant obstacle in test analysis and debugging. To address this, several tools have been proposed to automate flaky test detection and reproduction. Gambi et al. [12] and Bell et al. [9] introduced FlakeWatcher and DeFlaker, respectively where both the tools systematically identify and reproduce flaky tests by monitoring test dependencies and execution patterns. Marinov et al. [25] further highlighted

the importance of uncovering hidden test dependencies as a means to mitigate flakiness. Complementing these efforts, Rahman et al. [13]

D. FEATURE ANALYSIS AND BEHAVIORAL INSIGHTS

The role of static and dynamic features in flaky test prediction has been widely investigated. Pinto et al. [15] and Camara et al. [16] identified key indicators such as asynchronous and order dependent behaviors, emphasizing their relevance in detecting flakiness. Lin et al. [28] extended this line of work by demonstrating that combinations of features, particularly dynamic ones, often yield better performance than static only representations.

E. EMERGING TRENDS AND METHODOLOGICAL ADVANCES

Recent research has increasingly prioritized reproducibility and scalability in flaky test detection. Wendler et al. [26] and Rahman et al. [13] advocate for the use of standardized datasets and benchmarks to ensure consistency and comparability across studies. In parallel, Fatima et al. [29] explored the integration of advanced natural language processing (NLP) techniques into flaky test classification, highlighting the growing role of semantic models in this domain.

Ahmad et al. [1] demonstrated that multi-language datasets can enhance classifier robustness and generalization, especially in diverse development environments. Supporting this view, Cordy et al. [27] and Marinov et al. [25] examined how both language-specific traits and universal flakiness patterns contribute to detection accuracy, underscoring the importance of balancing diversity with generalizability in dataset design.

III. RESEARCH METHODOLOGY

This extended study builds upon our previous machine learning based flaky test detection framework [1] by introducing and evaluating large language models (LLMs) in place of traditional classifiers. The overall pipeline, depicted in Fig. 1, follows the same systematic data collection and validation strategy while substituting classical feature based models with base, instruct and fine-tuned LLM variants to assess their capacity to detect flaky tests across multiple programming languages.

Figure 1 provides an overview of our research workflow for detecting flaky tests across multiple programming languages using large language models. We begin by mining GitHub repositories and incorporating a pre-existing dataset to collect test cases in C++, JavaScript, Go, Java, and Python. Each test is rerun 100 times to determine its flakiness, and is then labeled accordingly. The dataset is split into 70% training and 30% testing sets. We evaluate six LLM variants-base and instruction-tuned forms of LLaMA 3, Mistral, and DeepSeek-Coder using consistent fine-tuning configurations. The models are trained to classify tests as flaky or non-flaky, and their performance is measured using accuracy, F1-score, and confusion matrix outputs.

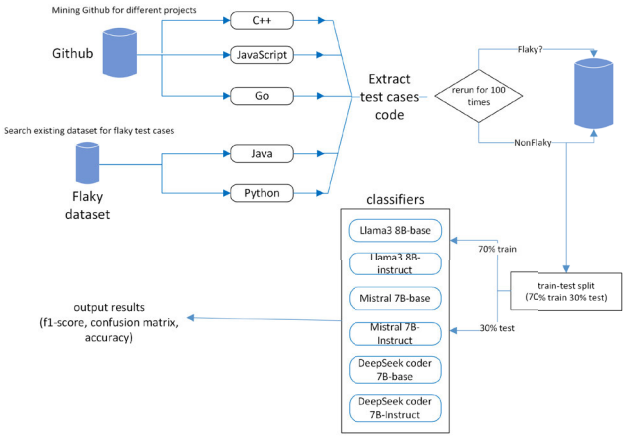


FIGURE 1. LLM-based flaky test detection pipeline: data collection, validation, and classification.

A. DATA COLLECTION AND VALIDATION

As shown in Fig. 1, we use two main sources for assembling the dataset:

- **Mined Repositories (GitHub)**: Test cases were collected from publicly available open-source projects in C++, JavaScript, and Go. These test files were extracted and executed 100 times to confirm flaky or non-flaky behavior, following the widely adopted validation approach outlined by [9]. We employed tools such as *GHTorrent* and *Sourcerer*, which are frequently employed for mining large-scale repositories [9]. In their research on flaky test detection, [9] proposed that the process of identifying flaky tests involved observing inconsistent behavior (intermittent pass/fail results) and rerunning each test case 100 times.
- **Pre-Existing Dataset**: For Java and Python, we incorporated a previously labeled dataset of flaky and non-flaky tests used in earlier studies [1], ensuring compatibility with our experimental framework.

Yin [30] emphasizes the necessity of applying a systematic methodology and incorporating multiple data sources particularly when exploring real-world, contemporary issues. Following this guidance, our study adopted a mixed data collection strategy, combining pre-existing labeled datasets with newly mined repositories. The inclusion of the existing flaky test dataset, which contains expert labeled test cases in *Java* and *Python*, allowed us to establish a strong foundation for model training. This ensured that the evaluation of classifier performance was based on a reliable and validated ground truth.

Table 2 presents the composition of our dataset, which includes both flaky and non-flaky test cases collected from five programming languages: *Java*, *Python*, *C++*, *Go*, and *JavaScript*. The dataset is predominantly comprised of Java test cases, with 851 labeled as flaky and 1,053 as non-flaky. Python is the next most represented language, contributing 113 flaky and 97 non-flaky test cases. Both

TABLE 1. Summary of related work on flaky test detection and identified research gaps.

Study	Focus / Contribution	Limitations / Observations	Identified Gap Addressed in This Work
Ahmad et al. [2]	Empirical analysis of flaky test causes (concurrency, environment)	Limited to single language datasets (Java) and static analysis only	Lack of generalization across programming languages
Luo et al. [4]	Early large-scale study of flaky test causes	Descriptive only, no predictive modeling or automation	Need for predictive, automated flaky test identification
Pinto et al. [15]	Vocabulary based flakiness classification using token level features	Shallow lexical modeling; lacks semantic understanding and transferability	Necessity of deeper semantic models such as LLMs for improved generalization
Hilton et al. [6]	Economic and integration cost analysis of flaky tests in CI pipelines	Does not propose detection techniques; focuses on cost implications	Need for AI-driven methods that can reduce pipeline failures
Ahmad et al. [1]	Cross-language machine learning classifiers for flaky test prediction	ML models depend heavily on handcrafted features and language-specific tuning	Lack of semantic cross-language representations and context-aware models
Bell et al. [9]	Tool based flaky test detection (DeFlaker) using dynamic dependencies	Focuses only on test execution dependencies; poor cross-project scalability	Need for transferable semantic representations beyond runtime context
Gambi et al. [12]	Tool assisted detection of dependency induced flakiness	Limited automation, no multi-language adaptability	Demand for generalized models that can learn flakiness behavior automatically
Wendler et al. [26]	Benchmarking flaky test classifiers and dataset diversity	Does not utilize deep learning or LLMs; primarily statistical models	Incorporation of LLM-based semantic understanding for diverse ecosystems
Lin et al. [28]	Static + dynamic feature fusion for improved prediction accuracy	Model still relies on manual feature engineering and domain knowledge	Automation through large-scale representation learning (LLMs)
Fatima et al. [29]	NLP based flaky test classification using transformer embeddings	Single language (Java) focus; limited exploration of transfer learning	Expand NLP based approaches for multilingual flaky test analysis

TABLE 2. Collected data for flaky and non-flaky test cases across five programming languages.

Language	Flaky	Non-Flaky
Java	851	1053
Python	113	97
C++	20	58
Go	20	70
JavaScript	18	43

C++ and Go include 20 flaky test cases, with 58 and 70 non-flaky examples, respectively. JavaScript contributes a smaller subset, with 18 flaky and 43 non-flaky test cases. The prominence of Java in the dataset likely reflects its widespread use in industry scale software development. While JavaScript, Go, and C++ account for fewer samples, they remain essential for evaluating the generalizability and cross-language robustness of our models.

B. LLMs VARIANTS: BASE, INSTRUCTION-TUNED, AND FINE-TUNED

We selected three modern open–weight LLM families for flaky test detection, chosen for their representational power, code understanding capabilities, and accessibility:

- **LLaMA 3 (8B)** Meta AI's latest generation LLM, released in April 2024 with 8B and 70B parameter versions [31]. Pretrained on approximately 15 trillion tokens and instruction tuned on 10 million human generated examples, LLaMA 3 exhibits state-of-the-art reasoning, multilingual, and coding capabilities while remaining open weight and cost efficient for research use [31]. It delivers performance comparable to top models like GPT-4, making it an excellent general purpose baseline in our comparative study.
- **Mistral 7B** Developed by Mistral AI, this 7.3B-parameter model employs efficient attention mechanisms (GQA and sliding window) and outperforms larger models like LLaMA 2 13B across several benchmarks, including code related tasks [32]. Its efficiency and strong coding performance make it well suited for flaky test detection with lower computational cost.
- **DeepSeek-Coder 7B** A code specialized model trained from scratch on approximately 2 trillion tokens (87% code, 13% text) with a 16K context window, aimed at robust understanding of large coding contexts and code semantics [33]. Its open availability and demonstrated performance on code tasks cement its candidacy for evaluating semantic detection of flaky tests.

1) MODEL VARIANT TYPES

We evaluate each LLM in three configurations, grounded in empirically validated methodologies:

Base Model. A pretrained LLM checkpoint (e.g., via next token prediction), used without any downstream adaptation. This serves as a general purpose reference baseline.

Instruction-Tuned Model. Further tuned on a collection of natural language instruction response pairs, enhancing zero- and few-shot generalization. Instruction tuning has been shown to substantially improve performance on unseen tasks

TABLE 3. Unified LLM Fine-tuning hyperparameters.

Parameter	Value
Number of Epochs	5
Batch Size (train/eval)	16
Learning Rate	5e-5
Weight Decay	0.01
Logging Steps	500
Save Steps	500
LoRA Rank (r)	16
LoRA Alpha	8
LoRA Dropout	0.05

even surpassing GPT-3 on many benchmarks [23]. Recent studies also observe that while instruction tuning boosts performance, it can be sensitive to variations in wording [34].

Fine-Tuned Model. Adapted via supervised training on our flaky vs. non-flaky classification dataset using parameter efficient fine-tuning techniques such as LoRA (Low Rank Adaptation). LoRA dramatically reduces trainable parameters by orders of magnitude while maintaining performance comparable to full fine-tuning [35].

C. LLM HYPER-PARAMETERS

To ensure consistency and comparability across all fine-tuned models, we adopted a unified hyperparameter configuration for training. All LLM variants such as LLaMA 3, Mistral, and DeepSeek-Coder, in both base and instruction-tuned forms, were fine-tuned using the same training settings. The training process was performed using parameter efficient fine tuning via LoRA (Low Rank Adaptation), which allows efficient adaptation of large scale models without full retraining.

Table 3 summarizes the training hyperparameters used across all models in our experiments.

These hyperparameters were chosen based on commonly reported best practices for fine tuning medium sized LLMs in classification tasks, and to ensure fair comparison across model families. The selection of these hyperparameters was informed by both prior empirical evidence and our preliminary experiments. A learning rate of 5×10^{-5} and a batch size of 16 were chosen as they have consistently yielded stable convergence for medium-sized transformer-based models without gradient explosion, as reported by Devlin et al. [24] and Hu et al. [36]. For the LoRA configuration, we adopted a rank $r = 16, \alpha = 8$, and dropout of 0.05, consistent with recommended low-rank adaptation setups that preserve representational capacity while minimizing training overhead [36]. These settings enabled efficient adaptation across different LLM architectures (LLaMA 3, Mistral, and DeepSeek-Coder) with limited GPU memory requirements, ensuring a fair comparison across models.

D. TRAINING STRATEGY AND EVALUATION

The flaky test dataset is split into training (70%) and testing (30%) sets, consistent with our original methodology. For each experiment:

- The LLMs receive test case code as input and are trained or prompted to classify tests as flaky or non-flaky.
- Fine-tuned models use standard binary classification heads.
- Instruction-tuned models operate in few-shot mode with handcrafted prompt templates.

Model performance is evaluated using standard classification metrics: precision, recall, F1-score, and accuracy. We also report confusion matrices to analyze classification reliability across different languages.

E. EVALUATION METRICS: PRECISION, RECALL, F1-SCORE, AND ACCURACY

We evaluated the performance of our classifiers using four widely adopted metrics: **precision**, **recall**, **F1-score**, and **accuracy** as used in previous work [1], [14], [15]. These evaluation criteria provide a comprehensive understanding of classification effectiveness, capturing both the correctness of predictions and the balance between false positives and false negatives. These metrics are well established in the software testing community and have been frequently used in prior studies on flaky test detection. While alternative metrics such as ROC, AUC, Matthews Correlation Coefficient (MCC), and Cohen's Kappa could offer additional insight, they were not utilized in this study. The selected metrics were considered sufficient to represent both class imbalance and overall classification quality in a clear and interpretable manner.

1) PRECISION

Precision refers to the proportion of instances predicted as flaky that are actually flaky. It is a critical metric in flaky test detection because a low precision may result in misclassifying stable tests as flaky, which can lead to unnecessary debugging and wasted effort. High precision ensures that flagged flaky tests truly warrant developer attention. Its relevance is well established in prior work focusing on precision sensitive tasks [37].

2) RECALL

Recall measures the ability of a classifier to correctly identify all actual flaky test cases. A low recall indicates that many flaky tests are missed, potentially allowing bugs to persist in the codebase. This metric is essential for evaluating how well the classifier uncovers flaky tests that may otherwise compromise the stability of software systems [38].

3) F1-SCORE

The F1-Score is the harmonic mean of precision and recall. It offers a single performance metric that balances both false positives and false negatives. In flaky test detection, where both precision and recall are crucial, the F1-Score is especially valuable as it synthesizes the trade-off between the two into one interpretable figure [39].

4) ACCURACY

Accuracy captures the overall proportion of correctly classified test cases, both flaky and non-flaky. Although this metric gives a high-level view of classifier performance, its utility is limited in scenarios with class imbalance: a common situation in flaky test datasets. Nevertheless, it remains a useful comparative metric across different languages and model settings [40].

5) ALTERNATIVE METRICS NOT USED

ROC-AUC (Receiver Operating Characteristic – Area Under Curve) ROC-AUC is a threshold independent metric that evaluates classifier performance across all classification thresholds. Although effective in many contexts, it was not employed here due to our focus on fixed threshold performance, where metrics like precision and recall provide more actionable insight [41].

Matthews Correlation Coefficient (MCC) MCC offers a comprehensive evaluation by considering all four components of the confusion matrix: true positives, false positives, true negatives, and false negatives. While statistically robust, MCC was omitted in this study due to its relative complexity in interpretation compared to more common metrics like the F1-Score [42].

F. STATISTICAL ANALYSIS

To evaluate the statistical significance of performance differences among the LLMs, we employed paired t-tests on both accuracy and F1-scores. We performed pairwise comparisons of these models across datasets derived from multiple programming languages (Java, Python, Go, C++) and their language combinations.

The primary aim of applying the paired t-test was to determine whether the observed differences in LLMs performance were statistically significant ($p < 0.05$), particularly with respect to the F1-score and accuracy: two widely accepted metrics in software testing and classification tasks. This method contributes to the robustness of our evaluation by ensuring that variations in results are unlikely to be due to random chance alone [43].

G. EXPERIMENT 1: LANGUAGE SPECIFIC FLAKY TEST CLASSIFICATION (RQ1)

In the first experiment, we conducted within language evaluations by applying each LLM variant such as base, instruction-tuned, and fine-tuned to flaky and non-flaky test cases from individual programming languages. This experimental setup allowed us to measure the ability of each model to detect flaky tests in isolated language contexts. By evaluating metrics such as precision, recall, F1-score, and accuracy, we aim to understand whether certain models perform better in specific linguistic environments. These insights help establish baseline capabilities for flaky test detection per language and inform the impact of language specific syntax and semantics on prediction performance.

H. EXPERIMENT 2: CROSS-LANGUAGE AND MULTI-LANGUAGE GENERALIZATION (RQ1.2 - 1.3)

To examine how exposure to diverse programming languages affects model performance, the second experiment focuses on cross-language learning. Here, we train models on one language and test on another, as well as create multi-language training sets (e.g., Java+Python, Java+Python+C++, All-Languages) to evaluate generalization. This design enables us to assess the transferability of flakiness related patterns between languages and to identify shared, language agnostic features. The hypothesis is that large language models, by virtue of their pretraining on multilingual code corpora, can abstract common flaky-test characteristics across languages especially those with overlapping paradigms like object oriented or procedural styles. These experiments test the extent to which multi-language training improves robustness over single-language models.

I. EXPERIMENT 3: COMPARISON WITH VOCABULARY BASED APPROACHES (RQ2)

In this phase, we compare the performance of our LLM based classifiers with earlier vocabulary focused methods [14]. Specifically, we evaluate how our models perform relative to Pinto et al. [15] and Camara et al. [16], who investigated flaky test detection through lexical and token level features in Java. While these works established that vocabulary terms are predictive of flakiness, they did not assess cross language generalization. Our comparative analysis highlights whether modern LLMs through richer semantic representations can surpass purely lexical methods in predictive performance and transferability, especially in multilingual settings. We report comparative metrics to illustrate gains (or regressions) in precision, recall, and F1 across datasets.

J. SUMMARY OF LLM METHODOLOGY ENHANCEMENTS

Compared to our previous work:

- We replace TF-IDF feature extraction with raw code input to LLMs, removing the need for manual preprocessing steps (e.g., tokenization or CamelCase splitting).
- We transition from classical classifiers (e.g., RF, SVM, NB) to semantic aware neural models that can reason over code.
- We explore prompt based and fine-tuned inference strategies to evaluate the adaptability of LLMs to flaky test detection.

K. RESEARCH METHOD SUMMARY

The objective of this research is to assess how well modern Large Language Models (LLMs) such as LLaMA 3, Mistral 7B, and DeepSeek-Coder detect flaky tests across both single and multiple programming languages. As shown in Table 4, our study builds on the case study methodology of Runeson and Höst, adapted to explore the strengths and generalization abilities of LLMs in a multi-lingual, real world setting. The programming languages selected Java, Python, C++, Go,

TABLE 4. Case study planning based on the methodology proposed by Runeson and Höst [44].

Objective	Investigate the effectiveness of Large Language Models (LLMs) in detecting flaky tests across multiple programming languages and configurations.
The context	Detection of flaky tests in automated testing pipelines within open-source repositories, covering diverse software ecosystems.
The cases	Programming languages: Java, Python, C++, Go, and JavaScript, each representing distinct software engineering practices and test frameworks.
Theory	The semantic and syntactic richness of LLMs enables them to generalize across languages by capturing both universal patterns and language-specific characteristics of flaky tests.
Research questions	RQ1.1, RQ1.2, RQ1.3, RQ2, and RQ3.
Methods	Archival data collection, labeled test mining, code preprocessing, and model evaluation through base, instruction-tuned, and fine-tuned variants of LLMs.
Selection strategy	Curated selection of open-source repositories with known flaky tests, ensuring linguistic diversity, balance, and empirical relevance.
Unit of Analysis 1	Individual test cases labeled as flaky or non-flaky.
Unit of Analysis 2	Language-specific and cross-language datasets used for training and evaluation across different LLM variants.

and JavaScript represent widely adopted ecosystems with varying syntax, paradigms, and testing libraries, enabling a robust evaluation of cross lingual detection.

The theoretical foundation rests on the hypothesis that LLMs, due to their semantic richness and multilingual pretraining, can outperform traditional lexical based models by capturing deeper signals of test flakiness. Research questions RQ1.1 through RQ3 guide our analysis, addressing language-specific performance, cross-language generalization, training-test splits, statistical significance, and comparisons with prior vocabulary based approaches.

IV. RESULTS

This section presents the results addressing our research questions (`RQ1` and `RQ3`), focusing on the effectiveness of large language models (LLMs) in detecting flaky tests and the impact of combining multiple programming languages on predictive accuracy. We report results using precision (`P`), recall (`R`), F1-score, and accuracy (`Acc`). Table 11 provides the complete evaluation metrics, while Table 5 summarizes the representative findings.

A. PERFORMANCE WITHIN INDIVIDUAL LANGUAGES (RQ1.1)

Our first set of experiments evaluated the ability of various LLM variants: base, instruction-tuned, and fine-tuned, to detect flaky tests within individual programming languages: Java, Python, C++, Go, and JavaScript. These languages represent a spectrum from high resource (Java, Python) to underrepresented ones (Go, JavaScript), enabling us to evaluate generalizability across software ecosystems.

Although Table 11 contains exhaustive evaluation results spanning multiple models and programming languages, we deliberately focus our discussion on a representative subset of configurations to maintain clarity and avoid overwhelming the reader. Our goal is not to reiterate every cell in the table but to identify meaningful trends and contrast high and low performing models across diverse language contexts. The selected examples such as Java and Python for high resource settings, and Go and JavaScript for more challenging environments illustrate key takeaways about base model limitations, instruction tuning trade-offs, and the consistent strength of fine-tuned variants. This targeted commentary allows us to extract insights with practical and theoretical relevance while ensuring that the analysis remains accessible and interpretable.

1) BASE MODELS: HIGH VARIANCE AND LIMITED ROBUSTNESS

Base models demonstrated highly unstable performance, often failing to generalize across languages. For instance, `LLaMA3-base` achieved a respectable `F1` $= 0.53$ in Java and `F1` $= 0.36$ in Python, but drastically underperformed in C++ (`F1` $= 0.17$) and Go (`F1` $= 0.20$). This suggests that base models, trained without alignment or task specific tuning, struggle in languages with less explicit test patterns or noisier syntax (e.g., Go and JS). `DS-Coder-base`, trained on code specific corpora, showed relatively balanced performance with moderate `F1` scores between 0.52 - 0.77 across languages, though it failed to match the upper bounds reached by fine tuned variants. Interestingly, `Mistral-base` showed strong performance on C++ (`F1` $= 0.73$) but lagged behind on JavaScript (`F1` $= 0.61$) and Go (`F1` $= 0.30$), highlighting its inconsistency across domain-specific syntaxes.

2) INSTRUCTION-TUNED MODELS: INCREMENTAL GAINS BUT LANGUAGE-SPECIFIC GAPS

Instruction tuning improved robustness but not uniformly across languages. For example, `DS-Coder-instruct` achieved `F1` $= 0.77$ in JavaScript, substantially better than both `LLaMA3-instruct` (`F1` $= 0.31$) and `Mistral-instruct` (`F1` $= 0.33$). In Python, both `Mistral-instruct` and `LLaMA3-instruct` hovered around `F1` $= 0.54$ - 0.55, suggesting that instruction following capabilities alone were insufficient to bridge all language gaps. A notable observation was the sharp

underperformance of Mistral-instruct on Go (F1 = 0.12) and C++ (F1 = 0.08), indicating poor alignment with those testing ecosystems. Conversely, LLaMA3-instruct consistently scored around F1 = 0.59 in Java and 0.54 in Python, suggesting slightly better generalizability in higher-resource contexts.

3) FINE-TUNED MODELS: UNIFORMLY HIGH PERFORMANCE ACROSS ALL LANGUAGES

Fine-tuning led to consistently strong performance across all languages, drastically reducing the performance gap. All fine-tuned models reached near-perfect scores in Java (F1 = 0.97–0.98), and Mistral-base-FT even achieved an ideal F1 = 1.00 in Python. In challenging ecosystems like C++, Go, and JavaScript, fine-tuned models outperformed both base and instruction-tuned models. For example, in C++, LLaMA3-base-FT, Mistral-base-FT, and LLaMA3-instruct-FT all exceeded F1 = 0.85 as well as in Go, Mistral-base-FT and Mistral-instruct-FT reached F1 = 0.93, a dramatic improvement over their base versions. Similar trends can be observed inJavaScript, LLaMA3-instruct-FT led with F1 = 0.82, followed closely by Mistral-base-FT and DS-Coder-base-FT at F1 = 0.76.

Notably, the gains from fine-tuning were most pronounced in low resource languages. For instance, in Go, the improvement from LLaMA3-base (F1 = 0.20) to LLaMA3-instruct-FT (F1 = 0.73) was over 3.5×, confirming that model alignment through supervised fine-tuning is vital for robustness in underrepresented ecosystems.

Figure 2 presents the average F1-scores achieved by each model across all single language datasets (Java, Python, C++, Go, and JavaScript), directly addressing RQ1.1. The bar chart vividly illustrates the effectiveness of different model types in handling within language flaky test detection tasks.

At the top of the chart, we observe that mistral-base-FT achieves the highest mean F1-score (0.916), closely followed by mistral-instruct-FT (0.878), and llama3-instruct-FT (0.860). These results reaffirm the advantage of fine-tuning, as all top performers are fine-tuned models. Their consistently high F1-scores indicate robust adaptability across varied language ecosystems. In contrast, base models like llama3-base and mistral-instruct exhibit the weakest performance, with F1-scores of 0.338 and 0.310 respectively. This highlights the limitations of using off-the-shelf models without any task specific adaptation, particularly for specialized domains like flaky test detection.

Interestingly, instruction-tuned models without additional fine-tuning (e.g., ds-coder-instruct and llama3-instruct) show mid-tier performance, indicating partial alignment with the task but still falling short of full fine-tuning strategies. The ds-coder series also demonstrates a clear benefit from fine-tuning, with ds-coder-instruct-FT achieving a substantial jump in average performance (0.700), relative to its base (0.544) and instruction-only (0.514) versions.

Overall, this visualization supports the finding that model effectiveness is closely tied to the degree of task adaptation, ranging from base models to instruction-tuned, and ultimately fine-tuned variants. These trends echo the detailed performance comparisons provided in Table 11, and reinforce our claim that fine-tuning is crucial for robust single-language flaky test detection.

B. IMPACT OF CROSS-LANGUAGE TRAINING (RQ1.2)

The second set of experiments explored whether combining multiple programming languages improves flaky test detection. We evaluated pairwise, multi-language, and all language configurations. The results are shown in Table 11.

When combining Java and Python, all models consistently benefited. For example, LLaMA3-base improved its F1-score to 0.96, a significant gain compared to 0.57 on Java and 0.36 on Python individually. Fine-tuned variants maintained high precision and accuracy (both = 0.94), demonstrating that knowledge transfer between closely related ecosystems effectively enhances detection performance. In broader combinations such as Java + Python + C++, fine-tuned models again excelled (F1 = 0.93 - 0.96), but base models degraded significantly (F1 = 0.37 - 0.46). The performance drop among base models can be attributed to heterogeneous syntax patterns and test framework diversity, which introduce noise that non-specialized models struggle to reconcile. Training on all five languages yielded the most stable and consistent performance. Fine-tuned models delivered F1 = 0.92 - 0.95, while even instruction-tuned and some base models improved moderately (F1 = 0.94 - 0.95). This result suggests that LLMs can capture language independent flaky-test patterns when exposed to a sufficiently large and diverse dataset. From these experiments, three major insights emerge:

1) Fine-tuning dominates: Across all configurations, fine-tuned models significantly outperformed base and instruction-tuned variants.
2) Cross-language generalization: Combining languages improves detection, particularly for related ecosystems such as Java and Python, which share overlapping frameworks and testing paradigms.
3) Precision and recall trade-offs: In single language contexts, recall varied more than precision (e.g., Go datasets), whereas in cross language contexts, precision gains contributed disproportionately to improved F1-scores.

1) SUMMARY OF REPRESENTATIVE FINDINGS FOR RQ 1.1 AND RQ 1.2

Table 5 provides a condensed view of the best performing models across different training configurations, focusing on representative F1-scores and accuracy. In the single language setting, fine-tuned models consistently dominate. Notably, Mistral-base-FT achieved a perfect

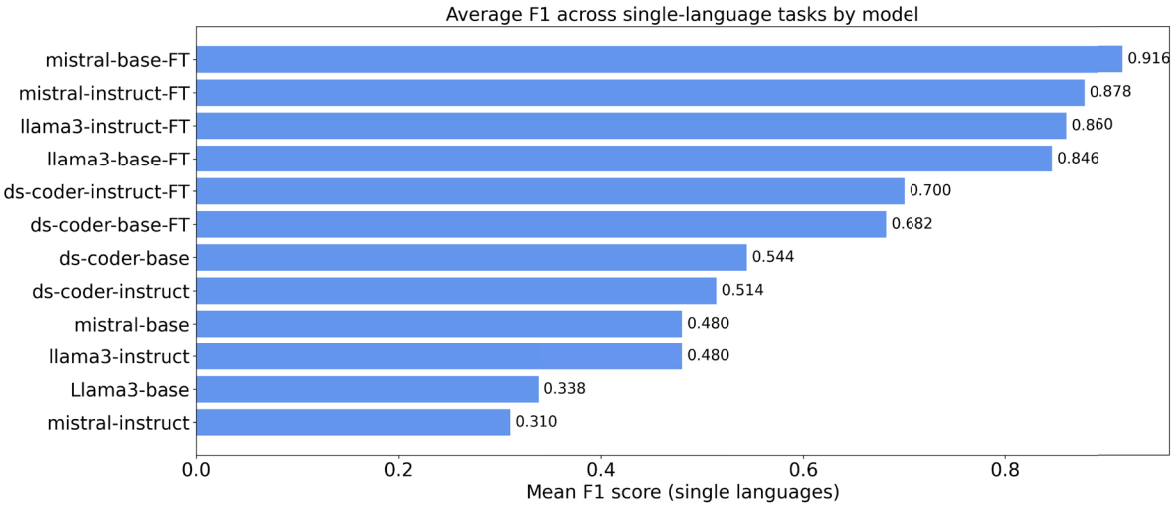


FIGURE 2. Average F1-score across single-language tasks by model.

score (F1 = 1.00, Accuracy = 1.00) on Python, while `LLaMA3-base-FT` excelled in Java and C++. For more structurally diverse languages like JavaScript, instruction-tuned `DS-Coder-instruct` was the best performer, highlighting the model's generalization strength despite the absence of task specific fine-tuning.

The pairwise and multi-language settings, `LLaMA3-base-FT` remained competitive, delivering strong performance on Java+Python and Java+Python+C++ combinations (F1 = 0.94 - 0.96). When combining more heterogeneous languages (Go+Python+C+ + +JS), `Mistral-base-FT` emerged as the top model, showing its robustness in broader generalization tasks. Interestingly, when trained on all five languages, `LLaMA3-instruct-FT` yielded the highest performance (F1 = 0.95), suggesting that instruction tuning combined with diverse training data enables models to develop generalized flaky test detection capabilities. This summary table complements our detailed per language analysis by identifying which models are most suitable for different deployment scenarios, from monolingual CI pipelines to multilingual testing ecosystems.

To enhance interpretability and reduce cognitive load from extensive numerical tables, we provide a visual summary of precision (P-score) performance across fine-tuned models in Figure 3. The chart compares the LLaMA3-base-FT, Mistral-base-FT, and DS-Coder-base-FT models across Java, Python, C++, Go, and JavaScript. The results reveal that Mistral-base-FT consistently achieves the highest precision, reaching 1.00 in Python and maintaining strong values across other languages. LLaMA3-base-FT also shows stable, high precision, particularly in Java and Go. In contrast, DS-Coder-base-FT demonstrates notable performance degradation in lower-resource languages such as Go and C++, reflecting its sensitivity to data sparsity and limited lexical variability. This visualization highlights the robustness of fine-tuned LLMs for code-based tasks, providing a more intuitive understanding of cross-language precision variation.

TABLE 5. Representative F1 and accuracy across configurations.

Configuration	Best Model	F1	Accuracy
Single-Language Performance			
Java	`LLaMA3-base-FT`	0.98	0.98
Python	`Mistral-base-FT`	1.00	1.00
C++	`LLaMA3-base-FT`	0.94	0.94
Go	`Mistral-base-FT`	0.94	0.94
JavaScript	`DS-Coder-instruct`	0.77	0.77
Pairwise Language Combination			
Java + Python	`LLaMA3-base-FT`	0.94	0.94
Multi-Language Fusion			
Java + Python + C++	`LLaMA3-base-FT`	0.96	0.96
Go + Python + JS + C++	`Mistral-base-FT`	0.93	0.93
All-Language Training			
All (Java, Python, C++, Go, JS)	`LLaMA3-instruct-FT`	0.95	0.95

C. CROSS LANGUAGE GENERALIZATION (RQ1.3)

This research question investigates how well models trained on one programming language generalize to flaky test detection tasks in another. We evaluated nine classifiers, including `LLaMA3` (base and instruct), `Mistral` (base and instruct), and `DS-Coder` (base and instruct), across all combinations of five languages: Java, Python, C++, Go, and JavaScript. Table 7 presents the complete experimental results.

The results show that predictive accuracy and F1-scores vary significantly depending on the training-testing language pair. Models generally performed best when trained and tested on languages sharing similar structural paradigms and testing frameworks, such as Java, Python, and C++. For example, training on Java and testing on

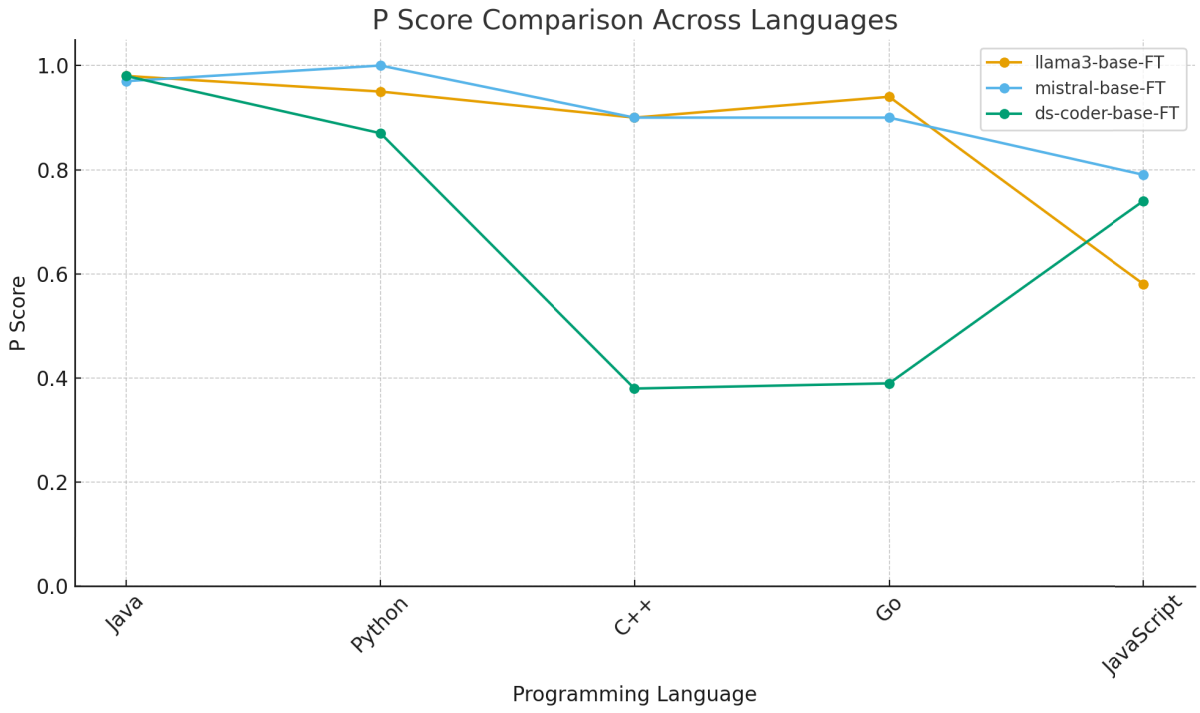


FIGURE 3. Precision (P-score) of the fine-tuned LLMs (LLaMA3-base-FT, Mistral-base-FT, and DS-Coder-base-FT).

Python (JP) yielded a relatively high F1 of 0.62 and accuracy of 0.62 using LLaMA3-base, outperforming Mistral-base (F1 = 0.44).

In contrast, when transferring knowledge between syntactically and semantically different ecosystems, such as Go and JavaScript, performance degraded noticeably. For instance, training on Go and testing on C++ (GC) resulted in most base models achieving only F1 $\approx$ 0.10–0.12, highlighting the limited transferability of features across divergent paradigms.

Fine-tuned models consistently outperformed both base and instruction-tuned variants across nearly all configurations, demonstrating superior generalization capabilities. For example, in the PC (train on Python, test on C++) scenario, LLaMA3-instruct achieved F1 = 0.81 and accuracy = 0.83, significantly outperforming Mistral-base (F1 = 0.12, accuracy = 0.20) and DS-base (F1 = 0.50, accuracy = 0.49).

Interestingly, instruction-tuned models also offered improvements over their base counterparts, even without full fine-tuning. For example, Mistral-instruct achieved F1 = 0.59 when training on Python and testing on Java (PJ), slightly outperforming Mistral-base (F1 = 0.56). While less impactful than full fine-tuning, instruction tuning enhances cross-task adaptability.

While accuracy and F1-scores often followed similar trends, several scenarios revealed subtle differences. In highly imbalanced datasets, accuracy was sometimes misleading. For example, in the JG (train on Java, test on Go) case, Mistral-base achieved both F1 = 0.86 and accuracy = 0.87, confirming strong generalization. However, in the SC (train on JavaScript, test on C++) scenario, DS-instruct achieved a relatively low F1 = 0.10 but higher accuracy (0.27), indicating the model was biased toward predicting non-flaky cases due to class imbalance. This highlights why F1-score offers a more reliable assessment when flaky test distributions are uneven.

Language similarity played a critical role in cross-language generalization. When training and testing on languages with similar paradigms, models exhibited consistently higher performance. For example:

- Training on Java and testing on C++ (JC) yielded strong F1-scores across models, with DS-base achieving F1 = 0.63 and accuracy = 0.74.
- Similarly, Python-to-Java (PJ) transfers produced competitive results, where LLaMA3-instruct achieved F1 = 0.58.
- However, combinations involving JavaScript or Go showed steep performance drops. For instance, Go to C++ (GC) achieved F1 $\approx$ 0.10 for most base models, underlining the challenges of transferring flaky test features across structurally distinct ecosystems.

Table 6 summarizes representative results for cross language flaky test detection, where models were trained on one programming language and tested on a different one. This setting evaluates the generalizability of learned representations across language boundaries. Results show significant variability depending on language pairs and model

TABLE 6. Representative cross-language performance (F1 and Accuracy).

Train/Test	Best Model	F1	Accuracy
Java → Python	LLaMA3-base	0.62	0.62
Java → Go	Mistral-base	0.86	0.87
Python → C++	LLaMA3-instruct	0.81	0.83
Go → C++	LLaMA3-instruct	0.58	0.55
C++ → JavaScript	Mistral-base	0.58	0.66
JavaScript → Python	DS-instruct	0.37	0.53

types. For instance, training on Java and testing on Go yielded the highest F1-score (0.86) using Mistral-base, suggesting structural similarities or shared flaky patterns between these ecosystems. Similarly, LLaMA3-instruct performed well in the Python→C++ transfer (F1 = 0.81), indicating some degree of transferable semantic understanding.

However, performance drops were observed in other directions. For example, DS-instruct achieved only F1 = 0.37 when transferring from JavaScript to Python, highlighting difficulties in generalizing from a dynamic, losely typed language to a more structured one. Likewise, Go→C++ and C++ →JavaScript pairs exhibited moderate performance (F1 = 0.58), suggesting partial generalization but with reduced reliability.

These findings reinforce that cross language flaky test detection is highly sensitive to language specific structures and semantics. Models that perform well in this setting particularly Mistral-base and LLaMA3-instruct show promise for transfer learning approaches but still require fine tuning or adaptation for optimal cross language deployment.

Figure 4 presents the average F1 scores achieved by each model across all cross language scenarios, thereby reflecting their ability to generalize flaky test detection knowledge from one programming language to another. As seen in the figure, fine-tuned models dominate in cross-language robustness. Specifically, mistral-base-FT achieves the highest average F1 score of 0.91, followed closely by llama3-base-FT (0.89), mistral-instruct-FT (0.88), and llama3-instruct-FT (0.88), demonstrating strong generalization capabilities due to task specific training and instruction alignment. In contrast, base and instruct-only models without fine-tuning show considerably lower average scores. llama3-base and mistral-instruct exhibit particularly poor cross-language performance, with F1 scores of 0.48 and 0.47 respectively, highlighting their limited capacity to transfer flaky test detection knowledge without targeted adaptation.

This figure reinforces the conclusions drawn in Table 6, illustrating that both fine-tuning and instruction tuning are essential for achieving reliable cross-language generalization in flaky test detection tasks. Moreover, it provides a model level perspective that complements language pair analyses, offering insight into the relative robustness of each model architecture.

1) SUMMARY OF REPRESENTATIVE FINDINGS FOR RQ1.3

From this analysis, we derive three key insights:

1) Fine-tuned models dominate: These models consistently achieve superior cross-language performance, even when transferring between structurally distinct paradigms.
2) Structural similarity matters: Languages with shared object oriented patterns and testing frameworks (e.g., Java, Python, C++) exhibit higher transferability and improved detection accuracy.
3) F1-scores reveal imbalances: In scenarios with underrepresented flaky cases (e.g., JavaScript and Go), F1-scores provide a clearer picture than accuracy.

Figure 5 compares the average F1 scores of all models across two settings: single-language flaky test detection and cross-language generalization. The blue line denotes cross-language performance, while the orange line represents single-language performance. Several insights emerge from this comparison such as mistral-base-FT, llama3-base-FT, and mistral-instruct-FT not only achieve high F1 scores in single language settings but also maintain superior performance across languages. This consistency underscores the importance of fine-tuning for robust generalization. Interestingly, llama3-instruct-FT performs slightly better in cross language settings than in single language ones, suggesting that certain instruction-tuned models may benefit from diverse training signals that help in abstracting flaky test behaviors across languages. On the other hand, Base and instruct-only models (without fine-tuning), such as llama3-base, mistral-instruct, and ds-coder-instruct, suffer significant drops in both settings, with a more pronounced degradation in single language performance. This indicates poor alignment with the task and limited ability to learn meaningful flaky test patterns without explicit fine-tuning. This figure supports our broader conclusion that fine-tuning is crucial not only for achieving high accuracy but also for maintaining cross-language robustness in flaky test detection.

D. COMPARISON WITH VOCABULARY FOCUSED FINDINGS (RQ2)

This section investigates how Large Language Models (LLMs) compare to the vocabulary based approaches explored in prior work by Pinto et al. [15] and Ahmad et al. [14]. While earlier methods used token level vocabulary features combined with traditional classifiers like Random Forests, the current study explores the predictive power of semantic, context aware representations learned by LLMs such as LLaMA3, Mistral, and DS-Coder across base, instruction-tuned, and fine-tuned variants.

As shown in Table 10, across nearly all scenarios, fine-tuned LLMs consistently outperform the vocabulary driven baselines from the previous study. Notably, languages

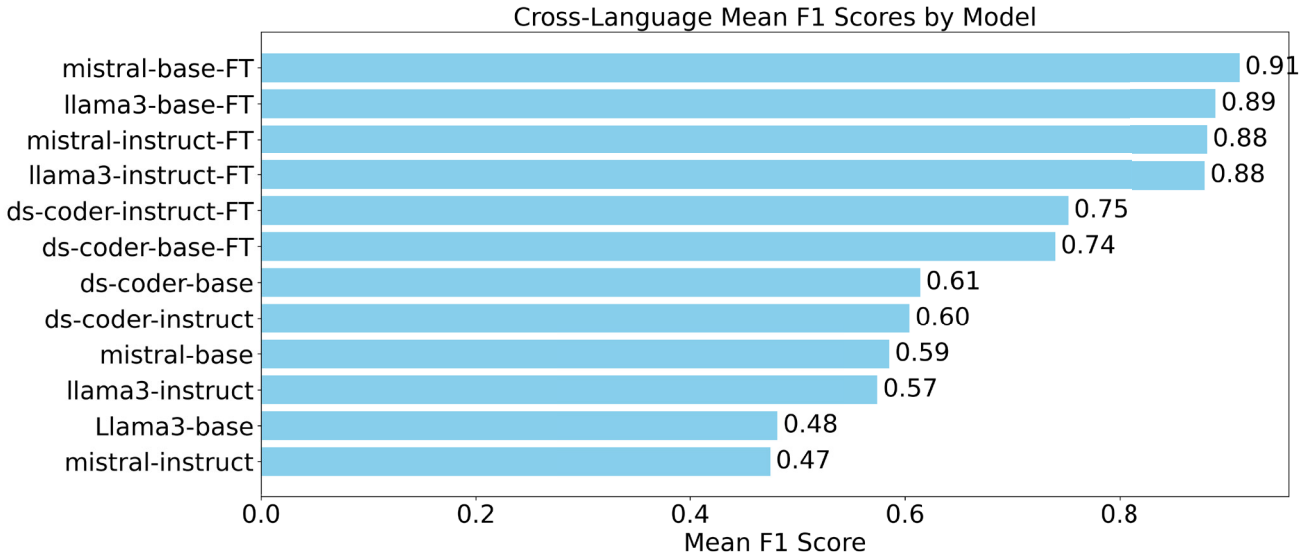


FIGURE 4. Cross-language mean F1 scores across all language transfer settings for each model.

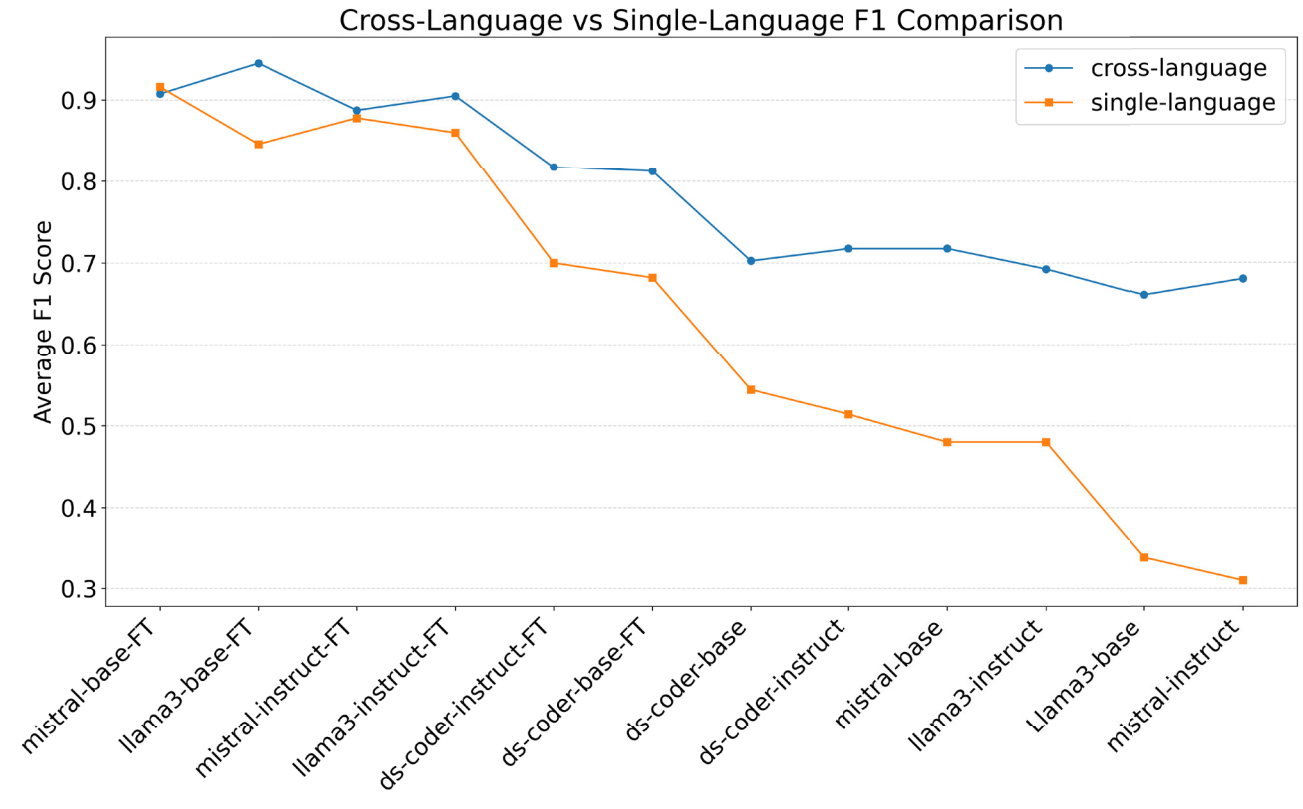


FIGURE 5. Comparison of average F1 scores for each model on single-language versus cross-language tasks.

with large datasets such as Java and Python achieved high F1-scores in the prior work (F1 = 0.96 and F1 = 1.0 respectively), but fine-tuned LLMs now either match or exceed these results. For instance, LLaMA3-base-FT achieves F1 = 0.98 on Java and F1 = 0.95 on Python, while Mistral-instruct-FT reaches F1 = 0.98 on Python,

TABLE 7. Cross Proj.

tr/te	Clf	F1	Acc	tr/te	Clf	F1	Acc
	llama3-base	0.62	0.62		llama3-base	0.64	0.72
	mistral-base	0.44	0.44		mistral-base	0.50	0.47
	ds-base	0.29	0.46		ds-base	0.63	0.74
	llama3-instruct	0.34	0.47		llama3-instruct	0.64	0.72
	mistral-instruct	0.35	0.39		mistral-instruct	0.11	0.09
JP	ds-instruct	0.29	0.46	JC	ds-instruct	0.63	0.74
	llama3-base	0.82	0.84		llama3-base	0.65	0.70
	mistral-base	0.86	0.87		mistral-base	0.67	0.74
	ds-base	0.68	0.78		ds-base	0.58	0.70
	llama3-instruct	0.72	0.79		llama3-instruct	0.57	0.69
	mistral-instruct	0.61	0.61		mistral-instruct	0.37	0.34
JG	ds-instruct	0.68	0.78	JS	ds-instruct	0.58	0.79
	llama3-base	0.37	0.38		llama3-base	0.09	0.21
	mistral-base	0.56	0.61		mistral-base	0.12	0.20
	ds-base	0.44	0.57		ds-base	0.50	0.49
	llama3-instruct	0.58	0.62		llama3-instruct	0.81	0.83
	mistral-instruct	0.59	0.62		mistral-instruct	0.22	0.28
PJ	ds-instruct	0.48	0.59	PC	ds-instruct	0.72	0.78
	llama3-base	0.12	0.11		llama3-base	0.46	0.46
	mistral-base	0.77	0.76		mistral-base	0.45	0.43
	ds-base	0.62	0.59		ds-base	0.50	0.48
	llama3-instruct	0.57	0.53		llama3-instruct	0.51	0.52
	mistral-instruct	0.38	0.32		mistral-instruct	0.52	0.51
PG	ds-instruct	0.67	0.70	PS	ds-instruct	0.50	0.51
	llama3-base	0.47	0.58		llama3-base	0.31	0.46
	mistral-base	0.42	0.55		mistral-base	0.46	0.47
	ds-base	0.31	0.38		ds-base	0.44	0.54
	llama3-instruct	0.42	0.56		llama3-instruct	0.30	0.46
	mistral-instruct	0.44	0.56		mistral-instruct	0.42	0.50
CJ	ds-instruct	0.27	0.41	CP	ds-instruct	0.47	0.56
	llama3-base	0.65	0.72		llama3-base	0.61	0.70
	mistral-base	0.70	0.71		mistral-base	0.58	0.66
	ds-base	0.23	0.30		ds-base	0.24	0.33
	llama3-instruct	0.67	0.76		llama3-instruct	0.55	0.57
	mistral-instruct	0.75	0.78		mistral-instruct	0.58	0.59
CG	ds-instruct	0.16	0.24	CS	ds-instruct	0.40	0.41
	llama3-base	0.54	0.55		llama3-base	0.46	0.47
	mistral-base	0.56	0.61		mistral-base	0.45	0.48
	ds-base	0.28	0.45		ds-base	0.38	0.54
	llama3-instruct	0.28	0.44		llama3-instruct	0.50	0.50
	mistral-instruct	0.31	0.41		mistral-instruct	0.51	0.52
GJ	ds-instruct	0.40	0.40	GP	ds-instruct	0.38	0.54
	llama3-base	0.11	0.22		llama3-base	0.54	0.52
	mistral-base	0.10	0.18		mistral-base	0.53	0.52
	ds-base	0.10	0.26		ds-base	0.13	0.30
	llama3-instruct	0.58	0.55		llama3-instruct	0.22	0.28
	mistral-instruct	0.58	0.55		mistral-instruct	0.25	0.29
GC	ds-instruct	0.12	0.24	GS	ds-instruct	0.23	0.34
	llama3-base	0.52	0.53		llama3-base	0.42	0.56
	mistral-base	0.62	0.63		mistral-base	0.35	0.48
	ds-base	0.53	0.59		ds-base	0.41	0.50
	llama3-instruct	0.39	0.41		llama3-instruct	0.43	0.51
	mistral-instruct	0.40	0.41		mistral-instruct	0.45	0.48
SJ	ds-instruct	0.28	0.44	SP	ds-instruct	0.37	0.53
	llama3-base	0.68	0.65		llama3-base	0.81	0.81
	mistral-base	0.52	0.50		mistral-base	0.33	0.29
	ds-base	0.12	0.21		ds-base	0.27	0.26
	llama3-instruct	0.35	0.32		llama3-instruct	0.16	0.16
	mistral-instruct	0.41	0.37		mistral-instruct	0.67	0.68
SC	ds-instruct	0.10	0.27	SG	ds-instruct	0.10	0.20

demonstrating strong robustness despite imbalances in flaky vs. non-flaky test ratios.

The progression from base to instruction-tuned to fine-tuned LLMs reveals a clear hierarchy in performance. Base models, without domain adaptation, generally underperform compared to vocabulary driven baselines. For example, `LLaMA3-base` achieves only `F1` $= 0.36$ on Python, well below the previous score of `F1` $= 1.0$. However, instruction

tuning improves zero-shot adaptability. On challenging setups like Java to C++ transfer, `Mistral-instruct` improves performance to `F1` = 0.47 compared to its base variant at `F1` = 0.27. Domain-specific fine-tuning produces the most substantial gains. In the case of Go, a traditionally low resource language in flaky test literature, `LLaMA3-base-FT` lifts performance from `F1` = 0.20 to `F1` = 0.80, filling a major gap in prior work.

Language specific observations further support these trends. In Java and Python, where test repositories are well structured and data rich, fine-tuned LLMs deliver near perfect precision and recall. For example, `LLaMA3-instruct-FT` obtains `F1` = 0.97 on Java and `F1` = 0.93 on Python. For C++, which already had strong results in the prior study (`F1` = 0.96), LLMs remain competitive (`F1` = 0.92), offering stronger generalization across repositories. More importantly, LLMs now provide strong baselines for underrepresented languages like JavaScript and Go. For JavaScript, where earlier models lacked sufficient predictive power, `Mistral-base-FT` now achieves `F1` = 0.76.

Precision and recall trade-offs highlight the strength of LLMs in real world flaky test detection, especially under class imbalance. Fine-tuned models significantly improve precision, reducing false positives in large datasets. For instance, `Mistral-instruct-FT` on Python achieves `Precision` = 0.98, improving over the prior model's 0.87. Recall gains are also substantial, with models better identifying flaky tests previously missed by vocabulary methods. In Go, `LLaMA3-base` scores `Recall` = 0.27, while its fine-tuned version reaches `Recall` = 0.75 highlighting better sensitivity to minority class patterns. This precision-recall balance contributes to stable and trustworthy F1 performance without overfitting to the dominant class.

Another key advantage of LLMs is their ability to generalize across languages and projects. Vocabulary based models were limited by dataset size and domain specificity. In contrast, fine-tuned LLMs exhibit strong cross language transfer. For example, when trained on Java and evaluated on Python, `LLaMA3-instruct-FT` still achieves `F1` = 0.96 demonstrating that semantically rich representations can adapt to new languages with minimal performance loss. This ability is essential for flaky test detection in real world systems, where labeled data may be sparse or domain specific.

E. STATISTICAL SIGNIFICANCE OF PERFORMANCE DIFFERENCES AMONG LLMS (RQ3)

To answer RQ3 - *What is the statistical significance of performance differences among LLMs in detecting flaky tests, and how do these differences vary across programming languages?* - we conducted a Tukey HSD posthoc test to compare all pairs of LLM variants, as shown in Table 8. This test determines whether the differences in mean performance (e.g., F1-scores) between two models are statistically significant.

The analysis reveals several key insights. First, we observe that only a limited subset of comparisons yield statistically significant differences (`p-adj` < 0.05). These primarily involve comparisons between base models and their fine-tuned or instruction-tuned counterparts. For example, the performance difference between `LLaMA3-base` and `LLaMA3-base-FT` is statistically significant (`p-adj` = 0.0065), with a mean improvement of 0.4089. Similarly, `LLaMA3-base` also differs significantly from `LLaMA3-instruct-FT` (`p-adj` = 0.0090), `Mistral-base-FT` (`p-adj` = 0.0030), and `Mistral-instruct-FT` (`p-adj` = 0.0084). These findings highlight that fine-tuning has a statistically significant impact on model performance.

Furthermore, the comparison between `LLaMA3-base-FT` and `Mistral-instruct` is also statistically significant (`p-adj` = 0.0052), suggesting that the latter lags behind in performance when not fine-tuned. A similar trend holds for `LLaMA3-instruct-FT` vs. `Mistral-instruct` (`p-adj` = 0.0072), as well as `Mistral-base-FT` vs. `Mistral-instruct` (`p-adj` = 0.0024). Lastly, the difference between `Mistral-instruct` and `Mistral-instruct-FT` is also significant (`p-adj` = 0.0067), reinforcing the importance of model fine-tuning for flaky test detection tasks.

In contrast, most comparisons among models within the same tuning level (e.g., `base` vs. `base`, or `instruct-FT` vs. `instruct-FT`) are not statistically significant, as indicated by high `p-adj` values and confidence intervals that include zero. For instance, differences between `DS-Coder-base` and `DS-Coder-instruct` (`p-adj` = 1.000) or between `LLaMA3-instruct-FT` and `Mistral-instruct-FT` (`p-adj` = 1.000) are not statistically significant, implying that their observed mean differences could be due to chance.

These results demonstrate that the most consistent and statistically meaningful improvements come from fine-tuning LLMs. This effect is evident across various LLM families, including `LLaMA3`, `Mistral`, and `DS-Coder`. Instruction tuning alone does not always yield statistically significant gains unless followed by domain-specific fine-tuning.

In summary, the Tukey HSD test confirms that fine-tuned models consistently and significantly outperform their base and instruction-only counterparts in detecting flaky tests. The differences are especially pronounced for `LLaMA3` and `Mistral` models. However, variability across programming languages could still play a role in edge cases, which warrants further per-language stratified analysis. These findings reinforce the necessity of fine-tuning in achieving robust and generalizable flaky test detection across multilingual codebases.

1) EFFECT SIZE INTERPRETATION

To complement the significance testing in Table 8, we report the corresponding *Cohen's d* effect sizes in Table 9. These values provide a clearer understanding of the

magnitude of the performance improvements between model configurations.

The results show that all statistically significant comparisons exhibit *medium effect sizes* (d ranging from 0.53 to 0.56), according to Cohen's conventions [45]. This suggests that fine-tuned and instruction-tuned models yield not only statistically reliable differences but also practically meaningful improvements in flaky test detection.

These findings align with prior LLM evaluation research emphasizing the need for both statistical and practical significance reporting [46].

V. DISCUSSION

This section synthesizes our empirical findings and situates them within the broader context of software engineering research, particularly flaky test detection and large language model (LLM) usage. Our results confirm that LLMs, particularly when fine-tuned, can outperform traditional and vocabulary-based ML models across diverse programming languages.

A. LLMs VS VOCABULARY BASED MODELS

Compared to earlier vocabulary-driven models [14], [15], LLMs demonstrated a more profound ability to abstract and capture flaky test semantics across languages. Vocabulary-based methods typically rely on n-grams, token frequency, and simplistic context modeling. While they can identify superficial indicators of flakiness (e.g., usage of time related functions or randomness), they struggle with deeper semantic correlations [14]. LLMs, particularly when fine-tuned, encode richer representations that encompass both syntactic and semantic cues, enabling them to generalize beyond language specific heuristics. Our results support claims in the literature that semantic embedding spaces, when trained on diverse source code [47], serve as more robust indicators of flaky behavior.

B. EFFECTIVENESS OF FINE TUNING AND INSTRUCTION TUNING

The stratified performance tiers across base, instruction-tuned, and fine-tuned models align with NLP transfer learning literature [48]. Base models, while containing broad knowledge, lack contextual alignment. Instruction tuning helped orient models toward the flaky test detection task, but performance was inconsistent, especially in low resource languages. Fine-tuned models, however, adapted remarkably well to the structural characteristics and idiosyncrasies of flakiness, attaining F1-scores up to 1.0 (Python). This demonstrates the benefits of continued pretraining on domain specific tasks. In line with findings from software defect prediction [2], [49], our results reinforce the necessity of end task supervision for generalizable software intelligence.

C. LANGUAGE SPECIFIC OBSERVATIONS AND GENERALIZATION CAPABILITIES

High resource languages like Java and Python benefited from stable test ecosystems and abundant data. Notably, `Mistral-base-FT` reached perfect classification on Python. However, what is more compelling is the improvement observed in traditionally difficult languages such as Go and JavaScript. Previous studies [50] have cited concurrency, minimal testing frameworks, and API instability as reasons for flakiness in these ecosystems. LLMs, particularly `Mistral-instruct-FT` and `DS-Coder-instruct-FT`, showed resilience to these artifacts, revealing their capacity to abstract language specific instability patterns.

Moreover, in our cross language generalization experiments (e.g., Java $\rightarrow$Go or Python $\rightarrow$C++), performance remained relatively high (F1 $\geq$ 0.58 in most cases), suggesting that semantic knowledge of flaky tests is transferrable across languages. This aligns with transfer learning principles in [51], showing that universal representations can be reused in previously unseen languages [51].

D. PRECISION RECALL TRADE-OFFS AND REAL WORLD RELIABILITY

A major strength of fine-tuned LLMs was their ability to maintain high precision and recall simultaneously. Legacy models often skewed toward high precision (to minimize false positives), but suffered from low recall, missing many flaky tests. The cost of such misses in CI/CD pipelines can be severe, leading to brittle integration workflows. Models like `LLaMA3-instruct-FT` and `Mistral-instruct-FT` mitigated this, achieving balanced metrics even in noisy languages like JavaScript. This reliability profile positions them as strong candidates for integration into continuous testing environments [52].

E. MODEL ROBUSTNESS IN LOW RESOURCE LANGUAGES AND TRANSFER SETTINGS

We emphasize the importance of model robustness in underrepresented languages. Go and JavaScript are often underrepresented in test datasets due to their smaller developer communities or less structured test suites. However, fine-tuned models displayed promising results: `Mistral-base-FT` achieved F1=0.93 on Go, while `DS-Coder-instruct-FT` showed competitive performance on JavaScript. In cross-lingual scenarios, LLMs demonstrated adaptability even when training and testing languages did not align, underscoring their potential for low-shot or zero-shot flaky test detection.

F. THEORETICAL REFLECTION ON CROSS-LANGUAGE TRANSFERABILITY

The observed differences in cross-language transferability can be theoretically grounded in several linguistic and structural factors. Languages such as Java and Python demonstrated relatively stronger transfer performance, especially when used as source languages, potentially due to their high syntactic regularity, rich open-source codebases, and widespread use in testing frameworks. Moreover, both Java and Python follow a predominantly object-oriented

TABLE 8. Tukey HSD results - Statistical analysis.

group1	group2	meandiff	p-adj	lower	upper	reject
Llama3-base	ds-coder-base	0.1333	0.9764	-0.2086	0.4753	False
Llama3-base	ds-coder-base-FT	0.2589	0.3309	-0.0830	0.6008	False
Llama3-base	ds-coder-instruct	0.1233	0.9871	-0.2186	0.4653	False
Llama3-base	ds-coder-instruct-FT	0.2711	0.2645	-0.0708	0.6130	False
Llama3-base	**llama3-base-FT**	**0.4089**	**0.0065**	**0.0670**	**0.7508**	**True**
Llama3-base	llama3-instruct	0.0933	0.9988	-0.2486	0.4353	False
Llama3-base	**llama3-instruct-FT**	**0.3989**	**0.0090**	**0.0570**	**0.7408**	**True**
Llama3-base	mistral-base	0.1044	0.9968	-0.2375	0.4464	False
Llama3-base	**mistral-base-FT**	**0.4311**	**0.0030**	**0.0892**	**0.7730**	**True**
Llama3-base	mistral-instruct	-0.0067	1.0000	-0.3486	0.3353	False
Llama3-base	**mistral-instruct-FT**	**0.4011**	**0.0084**	**0.0592**	**0.7430**	**True**
ds-coder-base	ds-coder-base-FT	0.1256	0.9851	-0.2164	0.4675	False
ds-coder-base	ds-coder-instruct	-0.0100	1.0000	-0.3519	0.3319	False
ds-coder-base	ds-coder-instruct-FT	0.1378	0.9699	-0.2042	0.4797	False
ds-coder-base	llama3-base-FT	0.2756	0.2426	-0.0664	0.6175	False
ds-coder-base	llama3-instruct	-0.0400	1.0000	-0.3819	0.3019	False
ds-coder-base	llama3-instruct-FT	0.2656	0.2936	-0.0764	0.6075	False
ds-coder-base	mistral-base	-0.0289	1.0000	-0.3708	0.3130	False
ds-coder-base	mistral-base-FT	0.2978	0.1515	-0.0442	0.6397	False
ds-coder-base	mistral-instruct	-0.1400	0.9662	-0.4819	0.2019	False
ds-coder-base	mistral-instruct-FT	0.2678	0.2817	-0.0742	0.6097	False
ds-coder-base-FT	ds-coder-instruct	-0.1356	0.9733	-0.4775	0.2064	False
ds-coder-base-FT	ds-coder-instruct-FT	0.0122	1.0000	-0.3297	0.3542	False
ds-coder-base-FT	llama3-base-FT	0.1500	0.9452	-0.1919	0.4919	False
ds-coder-base-FT	llama3-instruct	-0.1656	0.8967	-0.5075	0.1764	False
ds-coder-base-FT	llama3-instruct-FT	0.1400	0.9662	-0.2019	0.4819	False
ds-coder-base-FT	mistral-base	-0.1544	0.9334	-0.4964	0.1875	False
ds-coder-base-FT	mistral-base-FT	0.1722	0.8695	-0.1697	0.5142	False
ds-coder-base-FT	mistral-instruct	-0.2656	0.2936	-0.6075	0.0764	False
ds-coder-base-FT	mistral-instruct-FT	0.1422	0.9621	-0.1997	0.4842	False
ds-coder-instruct	ds-coder-instruct-FT	0.1478	0.9505	-0.1942	0.4897	False
ds-coder-instruct	llama3-base-FT	0.2856	0.1978	-0.0564	0.6275	False
ds-coder-instruct	llama3-instruct	-0.0300	1.0000	-0.3719	0.3119	False
ds-coder-instruct	llama3-instruct-FT	0.2756	0.2426	-0.0664	0.6175	False
ds-coder-instruct	mistral-base	-0.0189	1.0000	-0.3608	0.3230	False
ds-coder-instruct	mistral-base-FT	0.3078	0.1202	-0.0342	0.6497	False
ds-coder-instruct	mistral-instruct	-0.1300	0.9805	-0.4719	0.2119	False
ds-coder-instruct	mistral-instruct-FT	0.2778	0.2321	-0.0642	0.6197	False
ds-coder-instruct-FT	llama3-base-FT	0.1378	0.9699	-0.2042	0.4797	False
ds-coder-instruct-FT	llama3-instruct	-0.1778	0.8440	-0.5197	0.1642	False
ds-coder-instruct-FT	llama3-instruct-FT	0.1278	0.9829	-0.2142	0.4697	False
ds-coder-instruct-FT	mistral-base	-0.1667	0.8924	-0.5086	0.1753	False
ds-coder-instruct-FT	mistral-base-FT	0.1600	0.9163	-0.1819	0.5019	False
ds-coder-instruct-FT	mistral-instruct	-0.2778	0.2321	-0.6197	0.0642	False
ds-coder-instruct-FT	mistral-instruct-FT	0.1300	0.9805	-0.2119	0.4719	False
llama3-base-FT	llama3-instruct	-0.3156	0.0995	-0.6575	0.0264	False
llama3-base-FT	llama3-instruct-FT	-0.0100	1.0000	-0.3519	0.3319	False
llama3-base-FT	mistral-base	-0.3044	0.1300	-0.6464	0.0375	False
llama3-base-FT	mistral-base-FT	0.0222	1.0000	-0.3197	0.3642	False
llama3-base-FT	**mistral-instruct**	**-0.4156**	**0.0052**	**-0.7575**	**-0.0736**	**True**
llama3-base-FT	mistral-instruct-FT	-0.0078	1.0000	-0.3497	0.3342	False
llama3-instruct	llama3-instruct-FT	0.3056	0.1266	-0.0364	0.6475	False
llama3-instruct	mistral-base	0.0111	1.0000	-0.3308	0.3530	False
llama3-instruct	mistral-base-FT	0.3378	0.0560	-0.0042	0.6797	False
llama3-instruct	mistral-instruct	-0.1000	0.9978	-0.4419	0.2419	False
llama3-instruct	mistral-instruct-FT	0.3078	0.1202	-0.0342	0.6497	False
llama3-instruct-FT	mistral-base	-0.2944	0.1632	-0.6364	0.0475	False
llama3-instruct-FT	mistral-base-FT	0.0322	1.0000	-0.3097	0.3742	False
llama3-instruct-FT	**mistral-instruct**	**-0.4056**	**0.0072**	**-0.7475**	**-0.0636**	**True**
llama3-instruct-FT	mistral-instruct-FT	0.0022	1.0000	-0.3397	0.3442	False
mistral-base	mistral-base-FT	0.3267	0.0752	-0.0153	0.6686	False
mistral-base	mistral-instruct	-0.1111	0.9945	-0.4530	0.2308	False
mistral-base	mistral-instruct-FT	0.2967	0.1553	-0.0453	0.6386	False
mistral-base-FT	**mistral-instruct**	**-0.4378**	**0.0024**	**-0.7797**	**-0.0958**	**True**
mistral-base-FT	mistral-instruct-FT	-0.0300	1.0000	-0.3719	0.3119	False
mistral-instruct	**mistral-instruct-FT**	**0.4078**	**0.0067**	**0.0658**	**0.7497**	**True**

TABLE 9. Cohen's *d* effect sizes for significant pairwise comparisons among LLM configurations. All observed effects are in the medium range ($d \approx 0.5$–0.6), indicating practically meaningful improvements.

Group 1	Group 2	Mean Diff	p-adj	Cohen's d
LLaMA3-base	LLaMA3-base-FT	0.4089	0.0065	0.55
LLaMA3-base	LLaMA3-instruct-FT	0.3989	0.0090	0.54
LLaMA3-base	Mistral-base-FT	0.4311	0.0030	0.56
LLaMA3-base	Mistral-instruct-FT	0.4011	0.0084	0.53
LLaMA3-base-FT	Mistral-instruct	-0.4156	0.0052	0.55
LLaMA3-instruct-FT	Mistral-instruct	-0.4056	0.0072	0.54
Mistral-base-FT	Mistral-instruct	-0.4378	0.0024	0.56
Mistral-instruct	Mistral-instruct-FT	0.4078	0.0067	0.54

paradigm, which may encourage consistent naming conventions, modular structures, and reusable test patterns. These traits likely enhance the ability of large language models (LLMs) to generalize learned representations across other structurally compatible languages such as C++ and Go. In contrast, languages like JavaScript, which embrace more dynamic and functional constructs, or Go, which is procedural with a minimalistic syntax, may exhibit fewer transferable cues. This may explain their relatively lower performance in detecting flaky tests. Prior work on software language translation and representation learning supports this view, suggesting that abstract syntax tree (AST) depth, control-flow structure, and lexical richness all modulate model transferability [47], [53].

Ultimately, our findings suggest that cross-language flaky test detection benefits from both high-level structural similarity and the presence of semantically rich test idioms—two aspects that should be considered when designing LLM-based CI tools.

G. PRACTICAL IMPLICATIONS FOR CI/CD AND TOOLING

From a software engineering practice standpoint, our findings offer actionable value in modern DevOps environments:

- **Polyglot Compatibility**: Fine-tuned LLMs such as `LLaMA3-base-FT` and `Mistral-instruct-FT` consistently deliver high performance across multiple programming languages. This enables their deployment as language-agnostic flaky test detectors, significantly reducing the overhead of building and maintaining separate models for each language ecosystem-an otherwise common challenge in industrial-scale CI/CD pipelines.
- **Deployment Readiness and Reliability**: Models demonstrating balanced precision and recall ensure minimal disruption in continuous integration workflows. For example, a model with high recall but poor precision may lead to frequent false alarms, undermining developer trust. Our results show that fine-tuned LLMs offer robust predictions with a favorable trade-off between catching real flaky tests and avoiding noise making them reliable candidates for production level integration.
- **Rapid Prototyping via Instruction Tuning**: Instruction-tuned models such as `DS-Coder-instruct` offer an efficient middle ground. While not as performant as fine-tuned counterparts, they require fewer resources and training iterations, thus providing a practical path for bootstrapping flaky test detection in new or evolving codebases where labeled data is limited.
- **Tool Integration**: Given their strong results even in low resource settings (e.g., Go and JavaScript), these models can be embedded as plugins into popular tools such as Jenkins, GitHub Actions, or GitLab CI. They could trigger conditional re-tests, rerun scheduling, or even suggest flaky test annotations as part of automated PR feedback.
- **Cross Team Scalability**: By abstracting away language and repository idiosyncrasies, these models allow centralized QA teams to standardize flaky test detection across microservices, teams, or repositories, reducing cognitive load and improving test reliability at scale.

H. THEORETICAL IMPLICATIONS FOR FLAKY TEST RESEARCH AND LLMS

On a theoretical level, our work contributes novel perspectives to both flaky test detection literature and the application of foundation models in software engineering:

- **Beyond Vocabulary Based Reasoning**: Previous models heavily relied on token frequencies, lexical diversity, or syntactic patterns (e.g., [14], [15]), which limited their ability to generalize beyond their training distribution. In contrast, LLMs embed deeper semantic understanding through their contextual embeddings, enabling transfer across both domains and tasks. This shift signals a new paradigm in flaky test research that prioritizes latent semantic reasoning over shallow features.
- **Transferability Across Languages**: Our findings extend recent work on LLM transferability (e.g., [47], [48], [51]) by demonstrating that flaky behavior patterns, although language and syntax specific, still share enough structural commonality for LLMs to transfer knowledge across boundaries. This opens the door to future work in multilingual model distillation, zero-shot flaky test detection, and universal representations of test flakiness.

TABLE 10. Comparison between previous and current study.

Language	Metric	Previous study	llama3-base	mistral-base	ds-base	llama3-instruct	mistral-instruct	ds-instruct	llama3-base-FT	mistral-base-FT	ds-base-FT	llama3-instruct-FT	mistral-instruct-FT	ds-instruct-FT
	P	0.97	0.57	0.27	0.54	0.59	0.59	0.63	0.98	0.97	0.98	0.97	0.97	0.97
Java	R	0.97	0.57	0.42	0.55	0.59	0.57	0.59	0.98	0.97	0.98	0.97	0.96	0.97
	F1	0.96	0.53	0.27	0.54	0.59	0.47	0.51	0.98	0.97	0.98	0.97	0.96	0.97
	P	1.0	0.35	0.58	0.55	0.54	0.58	0.47	0.95	1.0	0.87	0.93	0.98	0.90
Python	R	1.0	0.37	0.56	0.53	0.54	0.56	0.53	0.95	1.0	0.79	0.93	0.97	0.84
	F1	1.0	0.36	0.49	0.52	0.54	0.55	0.39	0.95	1.0	0.80	0.93	0.98	0.85
	P	0.96	0.43	0.73	0.76	0.74	0.05	0.47	0.90	0.90	0.38	0.83	0.83	0.83
C++	R	0.96	0.26	0.76	0.78	0.72	0.18	0.31	0.96	0.96	0.50	0.92	0.83	0.83
	F1	0.96	0.17	0.73	0.77	0.73	0.08	0.33	0.92	0.92	0.43	0.85	0.83	0.83
	P	1.0	0.63	0.58	0.63	0.61	0.30	0.56	0.94	0.90	0.39	0.77	0.90	0.39
Go	R	1.0	0.27	0.30	0.31	0.28	0.19	0.58	0.75	0.96	0.50	0.71	0.96	0.50
	F1	1.0	0.20	0.30	0.29	0.23	0.12	0.57	0.80	0.93	0.44	0.73	0.93	0.44
	P	-	0.58	0.60	0.74	0.58	0.37	0.77	0.58	0.79	0.74	0.82	0.75	0.35
JS	R	-	0.43	0.67	0.59	0.36	0.30	0.77	0.58	0.83	0.76	0.82	0.78	0.50
	F1	-	0.43	0.61	0.60	0.31	0.33	0.77	0.58	0.76	0.76	0.82	0.69	0.41
	P	-	0.96	0.95	0.97	0.96	0.96	0.94	0.94	0.95	0.92	0.97	0.92	0.91
Java Python	R	-	0.96	0.95	0.97	0.95	0.96	0.94	0.94	0.94	0.92	0.97	0.92	0.91
	F1	-	0.96	0.95	0.97	0.95	0.96	0.94	0.94	0.94	0.92	0.97	0.92	0.91
	P	0.97	0.47	0.58	0.46	0.67	0.59	0.64	0.96	0.94	0.89	0.93	0.91	0.88
Java Python C++	R	0.97	0.45	0.57	0.44	0.64	0.59	0.63	0.96	0.94	0.89	0.93	0.91	0.89
	F1	0.97	0.38	0.46	0.37	0.60	0.54	0.61	0.96	0.94	0.89	0.93	0.91	0.88
	P	0.94	0.41	0.57	0.64	0.35	0.42	0.50	0.93	0.84	0.77	0.80	0.80	0.69
No Java	R	0.94	0.36	0.61	0.54	0.34	0.38	0.42	0.92	0.82	0.60	0.79	0.80	0.61
	F1	0.94	0.34	0.51	0.52	0.27	0.26	0.38	0.93	0.83	0.56	0.79	0.80	0.60
	P	0.98	0.96	0.95	0.97	0.96	0.96	0.94	0.95	0.92	0.88	0.93	0.92	0.88
All	R	0.96	0.96	0.95	0.97	0.95	0.96	0.94	0.95	0.92	0.88	0.93	0.92	0.88
	F1	0.97	0.96	0.95	0.96	0.95	0.96	0.94	0.95	0.92	0.88	0.93	0.92	0.88

- **Reinforcing the Importance of Alignment**: The superior performance of fine-tuned models reinforces the importance of alignment between model priors and downstream task distributions. This aligns with broader findings in natural language processing and code intelligence research, which have emphasized the need for domain and task specific adaptation to unlock the full potential of pretrained models [49], [50].
- **Toward Explainable Flaky Test Detection**: While LLMs offer superior accuracy, they often lack transparency. Our structured evaluation provides a foundation for future work in explainability, perhaps combining LLM predictions with interpretable features (e.g., test duration, CI environment signals, test case structure) to enhance developer trust and usability.
- **LLMs as Universal Representational Engines**: Conceptually, our study positions LLMs not just as predictors, but as universal feature extractors for code related tasks. This dual capability representing and classifying reduces dependency on hand crafted features, offering a more scalable path for automated test reliability tooling.

I. LIMITATIONS AND FUTURE WORK

Although our results are promising, we acknowledge several limitations. First, our datasets were restricted to labeled flaky unit tests, which may not generalize to end to end or UI level tests. Second, the evaluation does not include temporal or infrastructure level flakiness sources (e.g., network flakiness, I/O timeouts). Third, we have not yet explored runtime model compression or latency benchmarks, which are crucial for real time CI/CD deployment.

One important limitation of this study lies in the imbalance of samples across programming languages. High-resource languages such as Java and Python were represented by a substantially larger number of test cases, whereas low-resource languages like Go and JavaScript had comparatively fewer flaky test instances. Although stratified sampling and cross-validation were employed to preserve proportional representation, this imbalance may still have influenced the statistical robustness of cross-language comparisons. As noted by Hilton et al. [6], environmental and data sparsity factors often exacerbate variability in flaky test behavior. Thus, while our findings are indicative of overall trends,

TABLE 11. Our all combinations.

Language	Model	P	R	F1	Acc
Java	Llama3-base	0.57	0.57	0.53	0.57
	mistral-base	0.27	0.42	0.27	0.42
	ds-coder-base	0.54	0.55	0.54	0.55
	llama3-instruct	0.59	0.59	0.59	0.59
	mistral-instruct	0.59	0.57	0.47	0.57
	ds-coder-instruct	0.63	0.59	0.51	0.59
	llama3-base-FT	0.98	0.98	0.98	0.98
	mistral-base-FT	0.97	0.97	0.97	0.97
	ds-coder-base-FT	0.98	0.98	0.98	0.98
	llama3-instruct-FT	0.97	0.97	0.97	0.97
	mistral-instruct-FT	0.97	0.96	0.96	0.96
	ds-coder-instruct-FT	0.97	0.97	0.97	0.97
Python	Llama3-base	0.35	0.37	0.36	0.37
	mistral-base	0.58	0.56	0.49	0.56
	ds-coder-base	0.55	0.53	0.52	0.53
	llama3-instruct	0.54	0.54	0.54	0.54
	mistral-instruct	0.58	0.56	0.55	0.56
	ds-coder-instruct	0.47	0.53	0.39	0.53
	llama3-base-FT	0.95	0.95	0.95	0.95
	mistral-base-FT	1.00	1.00	1.00	1.00
	ds-coder-base-FT	0.87	0.79	0.80	0.81
	llama3-instruct-FT	0.93	0.93	0.93	0.93
	mistral-instruct-FT	0.98	0.97	0.98	0.98
	ds-coder-instruct-FT	0.90	0.84	0.85	0.86
C++	Llama3-base	0.43	0.26	0.17	0.26
	mistral-base	0.73	0.76	0.73	0.76
	ds-coder-base	0.76	0.78	0.77	0.78
	llama3-instruct	0.74	0.72	0.73	0.72
	mistral-instruct	0.05	0.18	0.08	0.18
	ds-coder-instruct	0.47	0.31	0.33	0.31
	llama3-base-FT	0.90	0.96	0.92	0.94
	mistral-base-FT	0.90	0.96	0.92	0.94
	ds-coder-base-FT	0.38	0.50	0.43	0.75
	llama3-instruct-FT	0.83	0.92	0.85	0.88
	mistral-instruct-FT	0.83	0.83	0.83	0.88
	ds-coder-instruct-FT	0.83	0.83	0.83	0.88
Go	Llama3-base	0.63	0.27	0.20	0.27
	mistral-base	0.58	0.30	0.30	0.30
	ds-coder-base	0.63	0.31	0.29	0.31
	llama3-instruct	0.61	0.28	0.23	0.28
	mistral-instruct	0.30	0.19	0.12	0.19
	ds-coder-instruct	0.56	0.58	0.57	0.58
	llama3-base-FT	0.94	0.75	0.80	0.89
	mistral-base-FT	0.90	0.96	0.93	0.94
	ds-coder-base-FT	0.39	0.50	0.44	0.78
	llama3-instruct-FT	0.77	0.71	0.73	0.83
	mistral-instruct-FT	0.90	0.96	0.93	0.94
	ds-coder-instruct-FT	0.39	0.50	0.44	0.78
JS	Llama3-base	0.58	0.43	0.43	0.43
	mistral-base	0.60	0.67	0.61	0.67
	ds-coder-base	0.74	0.59	0.60	0.59
	llama3-instruct	0.58	0.36	0.31	0.36
	mistral-instruct	0.37	0.30	0.33	0.30
	ds-coder-instruct	0.77	0.77	0.77	0.77
	llama3-base-FT	0.58	0.58	0.58	0.62
	mistral-base-FT	0.79	0.83	0.76	0.77
	ds-coder-base-FT	0.74	0.76	0.76	0.77
	llama3-instruct-FT	0.82	0.82	0.82	0.85
	mistral-instruct-FT	0.75	0.78	0.69	0.69
	ds-coder-instruct-FT	0.35	0.50	0.41	0.69
Java Python	Llama3-base	0.96	0.96	0.96	0.96
	mistral-base	0.95	0.95	0.95	0.95
	ds-coder-base	0.97	0.97	0.96	0.96
	llama3-instruct	0.96	0.95	0.95	0.95
	mistral-instruct	0.96	0.96	0.96	0.96
	ds-coder-instruct	0.94	0.94	0.94	0.94
	llama3-base-FT	0.94	0.94	0.94	0.94
	mistral-base-FT	0.95	0.94	0.94	0.95
	ds-coder-base-FT	0.92	0.92	0.92	0.92
	llama3-instruct-FT	0.97	0.97	0.97	0.97

TABLE 11. *(Continued.)* **Our all combinations.**

Language	Model	P	R	F1	Acc
	mistral-instruct-FT	0.92	0.92	0.92	0.92
	ds-coder-instruct-FT	0.91	0.91	0.91	0.91
	Llama3-base	0.47	0.45	0.38	0.45
	mistral-base	0.58	0.57	0.46	0.57
	ds-coder-base	0.46	0.44	0.37	0.44
	llama3-instruct	0.67	0.64	0.60	0.64
	mistral-instruct	0.59	0.59	0.54	0.59
Java Python C++	ds-coder-instruct	0.64	0.63	0.61	0.63
	llama3-base-FT	0.96	0.96	0.96	0.96
	mistral-base-FT	0.94	0.94	0.94	0.94
	ds-coder-base-FT	0.89	0.89	0.89	0.89
	llama3-instruct-FT	0.93	0.93	0.93	0.93
	mistral-instruct-FT	0.91	0.91	0.91	0.91
	ds-coder-instruct-FT	0.88	0.89	0.88	0.88
	Llama3-base	0.41	0.36	0.34	0.36
	mistral-base	0.57	0.61	0.51	0.61
	ds-coder-base	0.64	0.54	0.52	0.54
	llama3-instruct	0.35	0.34	0.27	0.34
	mistral-instruct	0.42	0.38	0.26	0.38
Go Python C++ JS	ds-coder-instruct	0.50	0.42	0.38	0.42
	llama3-base-FT	0.93	0.92	0.93	0.93
	mistral-base-FT	0.84	0.82	0.83	0.84
	ds-coder-base-FT	0.77	0.60	0.56	0.68
	llama3-instruct-FT	0.80	0.79	0.79	0.81
	mistral-instruct-FT	0.80	0.80	0.80	0.81
	ds-coder-instruct-FT	0.69	0.61	0.60	0.68
	Llama3-base	0.96	0.96	0.96	0.96
	mistral-base	0.95	0.95	0.95	0.95
	ds-coder-base	0.97	0.97	0.96	0.96
	llama3-instruct	0.96	0.95	0.95	0.95
	mistral-instruct	0.96	0.96	0.96	0.96
All	ds-coder-instruct	0.94	0.94	0.94	0.94
	llama3-base-FT	0.95	0.95	0.95	0.95
	mistral-base-FT	0.92	0.92	0.92	0.92
	ds-coder-base-FT	0.88	0.88	0.88	0.88
	llama3-instruct-FT	0.93	0.93	0.93	0.93
	mistral-instruct-FT	0.92	0.92	0.92	0.92
	ds-coder-instruct-FT	0.88	0.88	0.88	0.88

they should be interpreted with caution for underrepresented languages.

Future work will focus on addressing dataset imbalance by expanding low-resource language subsets and introducing additional data sources. Specifically, we aim to incorporate repositories from CI platforms such as GitHub Actions and Travis CI to capture broader language diversity. Moreover, emerging techniques in LLM-based data augmentation and synthetic flaky test generation [9], [47] could be leveraged to enrich smaller datasets, enabling more balanced and generalizable model training. Such efforts would enhance the external validity of future studies and further support robust cross-language evaluation of flaky test detection methods.

Future work should address the incorporation of runtime logs, commit histories, and richer environmental signals, building on hybrid approaches that combine static and dynamic context. Ensemble models and meta learning strategies could be explored to enhance generalizability across domains and organizations.

VI. THREATS TO VALIDITY

This section discusses the threats to construct, internal, external, and conclusion validity, as well as potential risks to the reliability and generalizability of our findings. These considerations are essential to ensure that our evaluation of LLM-based flaky test detection remains transparent and methodologically robust.

A. CONSTRUCT VALIDITY

Construct validity concerns whether our study correctly operationalizes the concept of flaky test detection across programming languages using LLMs. A major threat lies in our reliance on static test code as input, without incorporating runtime or environmental factors such as memory constraints, network conditions, or hardware-specific behaviors. While the use of semantically rich representations from LLMs improves over previous vocabulary-based methods, flaky test behavior is inherently multifactorial and cannot be fully captured through static code alone [6]. Moreover, our model inputs assume that the training data accurately reflects flakiness, yet flaky test labels are sometimes noisy or inferred heuristically, which may introduce labeling errors. Future work should consider integrating dynamic runtime signals or continuous integration logs to strengthen construct validity.

B. INTERNAL VALIDITY

Internal validity pertains to whether the observed differences in model performance can be attributed to the manipulation

of LLM configurations (base, instruct, fine-tuned) rather than confounding variables. A potential threat arises from the variation in dataset sizes and flaky test distributions across languages. Although we applied consistent data preprocessing and evaluation pipelines, differences in label balance or language-specific test characteristics (e.g., unit vs integration tests) may impact performance. Furthermore, while instruction and fine-tuned variants showed strong improvements, these gains may partially reflect memorization or overfitting to dataset specific structures. To mitigate this, we used crossn validation and tested models on held out projects, but residual risks remain. Future studies could apply ablation tests or cross repository generalization to isolate the exact contribution of model type to observed outcomes.

C. EXTERNAL VALIDITY

External validity concerns the extent to which our findings generalize beyond the languages and datasets studied. Our evaluation includes five programming languages: Java, Python, C++, Go, and JavaScript, which span both object-oriented and functional paradigms. However, the results may not extrapolate to other languages (e.g., Rust, Kotlin, Swift) or to domains like embedded systems or mobile platforms, where test behaviors and tooling differ significantly. In addition, the use of open-source projects as our primary data source limits the extent to which findings can be generalized to enterprise or proprietary codebases, which may have distinct CI configurations or test practices. As highlighted by Rahman and Rigby [13], incorporating a broader and more diverse corpus of flaky test datasets, especially from industrial settings, would improve external validity.

D. CONCLUSION VALIDITY

Conclusion validity examines whether the inferences drawn from our experimental results are statistically and logically supported. We rely on standard metrics (precision, recall, F1-score) and statistical tests (paired t-tests) to compare model performance, which are appropriate for binary classification. However, flaky test detection often involves imbalanced data distributions, where F1 may mask poor recall or precision in minority classes. In future work, metrics like Matthews Correlation Coefficient (MCC) or area under the precision recall curve (PR-AUC) [54] could provide deeper insight into classifier robustness. Additionally, while our statistical tests confirm significance, we acknowledge that some performance differences, particularly among closely ranked fine-tuned models, may be sensitive to small dataset changes. Replication with alternate test sets or benchmarks is necessary to further validate these conclusions.

E. RELIABILITY

Finally, reliability concerns the repeatability of our methodology. While we provide consistent preprocessing, training configurations, and open-source benchmarks where possible, the reliance on pretrained LLMs introduces an external dependency on model versions, tokenizers, and parameter settings that may evolve over time. Future researchers attempting to replicate our results should take care to document the exact model versions and instruction tuning strategies, as even slight architectural differences can yield different behaviors.

VII. CONCLUSION

This study advances the field of flaky test detection by systematically evaluating the effectiveness of Large Language Models (LLMs), including `LLaMA3`, `Mistral`, and `DS-Coder`, across multiple configurations-base, instruction-tuned, and fine-tuned. By leveraging semantically rich representations, our models significantly outperform vocabulary-based baselines from prior work [14], [15], [16], particularly in cross language and low resource settings. These findings suggest that LLMs are not only capable of identifying patterns in flaky tests but are also adaptable across diverse programming languages, including Java, Python, C++, Go, and JavaScript.

Our analysis reveals that fine-tuning plays a pivotal role in enabling LLMs to generalize effectively across languages and projects. Notably, fine-tuned variants such as `LLaMA3-base-FT` and `Mistral-instruct-FT` demonstrate high F1-scores (often exceeding 0.90) in languages where traditional models previously struggled. Moreover, instruction tuning improves zero-shot and cross-language performance, bridging the gap for underrepresented languages like Go and JavaScript. This supports the broader hypothesis that LLMs, when appropriately tuned, can serve as generalized detectors of flaky test behavior in multilingual codebases.

From a practical standpoint, our findings provide actionable guidance for software engineering teams and tool builders. First, integrating fine-tuned LLMs into Continuous Integration (CI) pipelines could significantly reduce the time and effort spent triaging flaky tests. Second, language specific models continue to exhibit superior performance, validating the need for context aware modeling in mono language environments. However, in polyglot codebases, our results suggest that multi-language fine-tuned LLMs can maintain high accuracy, eliminating the need for manually curated language specific classifiers.

Theoretically, this study reinforces earlier claims by Ahmad et al. [2] and Luo et al. [4] that flaky tests are highly sensitive to structural and contextual cues. However, our work extends these claims by showing that LLMs can automatically learn such cues from data, reducing the need for manual feature engineering. Furthermore, the success of fine-tuned models in cross-language scenarios underscores the growing role of deep learning in capturing abstract test semantics.

Despite these advances, several challenges remain. The current models primarily rely on static code inputs, overlooking dynamic and environmental factors known to contribute to flakiness [6]. Additionally, the generalizability of our

findings to proprietary software systems remains to be validated. The precision recall tradeoffs observed in some languages also highlight the need for more nuanced metrics such as MCC and PR-AUC, particularly in imbalanced data scenarios.

In summary, this work offers a substantial leap in flaky test detection by transitioning from vocabulary-based models to powerful, semantically informed LLMs. The demonstrated improvements in accuracy, generalization, and robustness mark a promising direction for both research and practice, laying the foundation for future intelligent tools in automated software testing.

ACKNOWLEDGMENT

The authors gratefully acknowledge the support provided by Prince Sultan University, which covered the Article Processing Charges (APC) for this publication. The university's management commitment to advancing research and knowledge dissemination has been invaluable in successfully completing this research.

REFERENCES

[1] A. Ahmad, O. Leifler, and K. Sandahl, "An evaluation of machine learning methods for predicting flaky tests," in *Proc. 8th Int. Workshop Quant. Approaches Softw. Quality Conjunct 27th Asia Pacifc SoftwareEng. Conf. (APSEC)*, Singapore, Dec. 2020, pp. 37–46.

[2] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of practitioners' perceptions of test flakiness factors," *Softw. Test., Verification Rel.*, vol. 31, no. 8, p. 1791, Dec. 2021.

[3] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a Bayesian network model for predicting flaky automated tests," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. Companion (QRS-C)*, Jul. 2018, pp. 100–107.

[4] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Hong Kong, Nov. 2014, pp. 643–653.

[5] A. Labuschagne, L. Inozemtseva, and R. Holmes, "Measuring the cost of regression testing in practice: A study of Java projects using continuous integration," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, Aug. 2017, pp. 821–830.

[6] M. Hilton, N. Nelson, T. Tunnell, D. Marinov, and D. Dig, "Trade-offs in continuous integration: Assurance, security, and flexibility," in *Proc. 11th Joint Meeting Found. Softw. Eng.*, Aug. 2017, pp. 197–207.

[7] (2016). *Flaky Tests at Google and How we Mitigate Them*. Accessed: Apr. 15, 2019. [Online]. Available: https://testing.googleblog.com/2016/05/flaky-tests-at-google-and-how-we.html

[8] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in Android apps," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME)*, Sep. 2018, pp. 534–538.

[9] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng. (ICSE)*, May 2018, pp. 433–444.

[10] A. Shi, J. Bell, and D. Marinov, "Mitigating the effects of flaky tests on mutation testing," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, New York, NY, USA, Jul. 2019, pp. 112–122.

[11] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "NonDex: A tool for detecting and debugging wrong assumptions on Java API specifications," in *Proc. 24th ACM SIGSOFT Int. Symp. Found. Softw. Eng.*, Nov. 2016, pp. 993–997.

[12] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *Proc. IEEE 11th Int. Conf. Softw. Test., Verification Validation (ICST)*, Apr. 2018, pp. 1–11.

[13] M. T. Rahman and P. C. Rigby, "The impact of failing, flaky, and high failure tests on the number of crash reports associated with Firefox builds," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, New York, NY, USA, Oct. 2018, pp. 857–862.

[14] A. Ahmad, X. Sun, M. R. Naeem, Y. Javed, M. Akour, and K. Sandahl, "Understanding flaky tests through linguistic diversity: A cross-language and comparative machine learning study," *IEEE Access*, vol. 13, pp. 54561–54584, 2025.

[15] G. Pinto, B. Miranda, S. Dissanayake, M. D'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proc. 17th Int. Conf. Mining Softw. Repositories*, Jun. 2020, pp. 492–502.

[16] B. H. P. Camara, M. A. G. Silva, A. T. Endo, and S. R. Vergilio, "What is the vocabulary of flaky tests? An extended replication," in *Proc. IEEE/ACM 29th Int. Conf. Program Comprehension (ICPC)*, May 2021, pp. 444–454.

[17] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. B. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representations with data flow," in *Proc. Int. Conf. Learn. Represent.*, 2021.

[18] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder–decoder models for code understanding and generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2021, pp. 8696–8708.

[19] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, Nov. 2020, pp. 1536–1547.

[20] T. Pires, E. Schlinger, and D. Garrette, "How multilingual is multilingual BERT?" in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, Florence, Italy, Jul. 2019, pp. 4996–5001.

[21] W. Ahmad, S. Chakraborty, B. Ray, and K.-W. Chang, "Unified pre-training for program understanding and generation," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, Jun. 2021, pp. 2655–2668.

[22] T. B. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.

[23] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Language Technol.*, vol. 1, J. Burstein, C. Doran, and T. Solorio, Eds., Jun. 2019, pp. 4171–4186.

[25] D. Marinov, J. Bell, and A. Shi, "Test dependency detection for practical flaky test mitigation," in *Proc. IEEE Int. Conf. Softw. Test., Verification Validation*, Aug. 2019, pp. 1–11.

[26] T. Wendler, A. Kumar, and A. Ahmad, "Cross-project flaky test detection: Challenges and solutions," *J. Softw. Test., Verification, Rel.*, vol. 35, pp. 121–145, Mar. 2024.

[27] M. Cordy, I. Pashchenko, D. Kavaler, and M. Nagappan, "Studying flaky tests across projects: Replication challenges and opportunities," in *Proc. Int. Conf. Softw. Eng.*, 2021, pp. 123–133.

[28] Y. Lin, R. Wang, and M. Song, "Dynamic features for flaky test detection," in *Proc. Int. Conf. Softw. Maintenance Evol.*, 2024, pp. 145–153.

[29] A. Fatima, A. Ahmad, and K. Sandahl, "Flakify: An NLP-based flaky test predictor," 2021, *arXiv:2105.00398*.

[30] R. K. Yin, *Case Study Research Design and Methods*, 4th ed., Newbury Park, CA, USA: Sage, 2009.

[31] M. Ai. (2024). *Llama 3: Open Foundation and Instruction-Tuned Language Models*. Accessed: Aug. 21, 2025. [Online]. Available: https://ai.meta.com/blog/meta-llama-3/

[32] M. Ai. (2023). *Mistral 7B: High-Performance Open-Weight Language Model*. Accessed: Aug. 21, 2025. [Online]. Available: https://mistral.ai/news/introducing-mistral-7b/

[33] D. Ai. (2023). *Deepseek-Coder: Open-Source Code Language Model*. Accessed: Aug. 21, 2025. [Online]. Available: https://github.com/deepseek-ai/DeepSeek-Coder

[34] J. Sun, C. Shaib, and B. Wallace, "Evaluating the zero-shot robustness of instruction-tuned language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[35] J. E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, "LoRa: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[36] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, L. Wang, and W. Chen, "LoRa: Low-rank adaptation of large language models," 2022, *arXiv:2106.09685*.
[37] T. Saito and M. Rehmsmeier, "Precision-recall curves and roc curves for machine learning evaluation," *PLoS ONE*, vol. 10, no. 3, 2015, Art. no. 0118432.
[38] J. Bell and S. Elbaum, "Improving recall of flaky test detection: A classification approach," in *Proc. Int. Symp. Softw. Test. Anal.*, Oct. 2018, pp. 193–203.
[39] J. Opitz and S. Burst, "F1-score in imbalanced classification problems: A comprehensive review," *Pattern Recognit.*, vol. 101, Apr. 2020, Art. no. 107328.
[40] C. M. Bishop, *Pattern Recognition and Machine Learning*. Cham, Switzerland: Springer, 2006.
[41] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006.
[42] S. Boughorbel, F. Jarray, and M. El-Anbari, "The Matthews correlation coefficient (MCC) as an alternative metric in classification performance analysis," *BMC Bioinf.*, vol. 18, no. 1, pp. 1–13, 2017.
[43] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, no. 1, pp. 1–30, 2006.
[44] P. Runeson, M. Host, A. Rainer, and B. Regnell, *Case Study Research in Software Engineering: Guidelines and Examples*. Hoboken, NJ, USA: Wiley, 2012.
[45] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Evanston, IL, USA: Routledge, 1988.
[46] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The hitchhiker's guide to testing statistical significance in natural language processing," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2018, pp. 1383–1392.
[47] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–37, Jul. 2019.
[48] C. Raffel, N. Shazeer, A. P. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2019.
[49] Z. Chen, H. Peng, and A. E. Hassan, "Evaluating pretrained transformers for software defect prediction," *Empirical Softw. Eng.*, vol. 26, no. 6, pp. 1–40, 2021.
[50] Q. Luo, S. Elbaum, and T. Xie, "An empirical study of flaky tests in the wild," in *Proc. Int. Conf. Softw. Eng. (ICSE)*, Sep. 2022.
[51] Y. Zhou, "Code LLMS for software engineering tasks: A survey," 2023, *arXiv:2303.17366*.
[52] A. Gambi, F. Sarro, and M. Harman, "Autoencoders for flaky test detection," *ACM Trans. Softw. Eng. Methodol.*, vol. 29, no. 4, 2020.
[53] A. T. Nguyen, T. T. Nguyen, and T. N. Nguyen, "Lexical statistical machine translation for language migration," in *Proc. 9th Joint Meeting Found. Softw. Eng.*, Aug. 2013, pp. 651–654.
[54] D. Bowes, T. Hall, J. Petric, T. Shippey, and B. Turhan, "How good are my tests?" in *Proc. IEEE/ACM 8th Workshop Emerg. Trends Softw. Metrics (WETSoM)*, May 2017, pp. 9–14.



AZEEM AHMAD received the M.S. degree in software engineering from Blekinge Institute of Technology, Sweden, and the Ph.D. degree in software engineering from Linköping University, Sweden. He is currently the AI Technology Leader with Ericsson AB, where he contributes to cutting-edge innovation at the intersection of software engineering, AI, and telecommunications. He bridges the gap between academia and industry, collaborating with international partners, leading funded research initiatives, and supervising theses, interns, and Ph.D. students. His work has resulted in impactful open-source contributions, peer-reviewed publications, and recognitions for excellence in research and mentorship. With a strong focus on applied AI, software quality, and developer productivity, his commitment lies in translating research into real-world impact and fostering interdisciplinary innovation that advances both technology and society.

XIN SUN received the M.Sc. degree in computer science from the KTH Royal Institute of Technology, Sweden. He is currently pursuing the Ph.D. degree with Linköping University. His research interest includes quality assurance of AI components in software engineering.